TruthLens: An Evidence-Oriented Multimodal Platform for Detecting AI-Generated Media with Proctored Identity Verification

Project Report

Submitted in fulfillment of the requirements

for the degree of

Bachelor of Technology

in

Information Technology

by

Manini Jaiswal (23101C0060)
Rudrapratap Shinde (23101B0015)
Soham Thakur (23101B0012)
Atharva Pethkar (23101B0010)

Project Guide

Prof. Dhanashree Tamhane

Department of Information Technology
Vidyalankar Institute of Technology
Wadala (E), Mumbai – 400037
(Affiliated to University of Mumbai)

2026-2027

Certificate

This is to certify that the project entitled

"TruthLens: An Evidence-Oriented Multimodal Platform for Detecting AI-Generated Media with Proctored Identity Verification"

is a bonafide work done by

Manini Jaiswal (23101C0060)

Rudrapratap Shinde (23101B0015)

Soham Thakur (23101B0012)

Atharva Pethkar (23101B0010)

and is submitted in fulfillment of the requirement for the degree of

Bachelor of Technology

in Information Technology

to the

Vidyalankar Institute of Technology, Wadala, Mumbai.

Project Guide	Head of Department	Principal

Dissertation Approval for B.Tech.

This is to certify that the dissertation entitled "**TruthLens: An Evidence-Oriented Multimodal Platform for Detecting AI-Generated Media with Proctored Identity Verification**" is a bonafide work done by **Manini Jaiswal, Rudrapratap Shinde, Soham Thakur and Atharva Pethkar** under the supervision of **Prof. Dhanashree Tamhane**. This dissertation has been approved for the award of the B.Tech. degree in Information Technology, Vidyalankar Institute of Technology, Wadala, Mumbai.

Examiners:

1. _____

2. _____

Date: _____

Place: _____

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

1. _____

2. _____

3. _____

4. _____

(Roll No., Name and Signature of Students)

Date: _____

Place: _____

Acknowledgement

We express our sincere gratitude to our project guide, **Prof. Dhanashree Tamhane**, for her constant guidance, patient reviews and the many suggestions that shaped this work from the first proposal to the final review.

We thank the Head of the Department of Information Technology and the Principal of Vidyalankar Institute of Technology for providing the facilities and the encouragement needed to carry out a project of this scope. We are also grateful to the faculty members and laboratory staff of the department for their help during the reviews and for access to the laboratory resources.

We acknowledge the authors and maintainers of the public datasets and open-source libraries this project builds on, including CIFAKE, DeepDetect-2025, RealTimeFaceSwap-10k, FaceForensics++, Celeb-DF, DiffusionDB, PyTorch, OpenCLIP, Hugging Face Transformers, facenet-pytorch, the C2PA SDK and FastAPI. Free GPU time on Kaggle and Google Colab made the fine-tuning rounds possible.

Finally, we thank our families and friends for their support and patience throughout the project.

Manini Jaiswal (23101C0060)
Rudrapratap Shinde (23101B0015)
Soham Thakur (23101B0012)
Atharva Pethkar (23101B0010)

Abstract

Image generators, face-swap apps and voice-cloning services have made synthetic media cheap to produce and hard to recognise by eye. This project presents TruthLens, a working web platform that examines an uploaded image, video or audio file and returns a banded verdict (REAL, UNCERTAIN or FAKE) together with the evidence behind it. Images are scored by a fine-tuned ConvNeXt-Tiny classifier, a frozen CLIP ViT-B/16 probe and, when a face is present, a face-swap check, combined by a maximum rule; videos by an EfficientNet-B0 applied to sixteen face crops with Platt-calibrated scores and a resolution guard; speech by a fine-tuned wav2vec 2.0 model over 4-second windows. Every verdict comes with Grad-CAM maps or per-frame and per-window scores, C2PA Content Credential validation, metadata and Stable Diffusion watermark checks, calibrated confidence and a PDF report. A second module, ProctorLens, verifies a candidate against a government ID and monitors proctored online examinations with eight behavioural signals. The deployed models were evaluated separately on data sources seen and unseen in training: the image ensemble reaches 98.4% accuracy on 4,000 seen-source images but 53.2% on 2,517 images from unseen generators; the round-4 video model raises held-out ROC-AUC from 0.765 to 0.961; identity matching reaches 96.5% accuracy on 1,000 LFW pairs. The results show that detector scores must be presented as one strand of evidence rather than a final answer.

Keywords: Deepfake detection, AI-generated images, multimodal media forensics, explainable AI, content provenance, online proctoring

Contents

Abstract vi

List of Figures ix

List of Tables xii

Abbreviations xiii

Chapter 1 Introduction 1
1.1 Background 1
1.2 Motivation 2
1.3 Problem Statement 2
1.4 Objectives 3
1.5 Scope of the Work 3
1.6 Organization of the Report 4

Chapter 2 Literature Survey 5
2.1 Review of Existing Work 5
2.1.1 AI-Generated Image Detection 5
2.1.2 Face-Manipulation Video Detection 5
2.1.3 Synthetic Speech Detection 6
2.1.4 Explanation, Provenance and Calibration 6
2.1.5 Face Verification and Online Proctoring 6
2.2 Summary of Literature 7
2.3 Research Gaps 7

Chapter 3 Proposed Methodology 9
3.1 System Architecture 9
3.2 Dataset Description 10
3.3 Algorithm / Model Description 12
3.3.1 Image Pipeline 12
3.3.2 Video Pipeline 14
3.3.3 Audio Pipeline 15
3.3.4 Verdict Bands and Calibrated Confidence 16
3.3.5 Explanation Layer 16
3.3.6 Provenance Layer 17
3.3.7 ProctorLens: Identity Verification and Exam Monitoring 18
3.3.8 Performance Metrics 19
3.4 Tools and Technologies Used 20

Chapter 4 System Design and Implementation 21
4.1 Requirements 21
4.1.1 Functional Requirements 21
4.1.2 Non-Functional Requirements 21

4.1.3 Hardware and Software Requirements 22
4.2 Use-Case Diagram 23
4.3 Data Flow Diagrams 23
4.4 Sequence Diagrams 24
4.5 Database Design 26
4.6 API Design 26
4.7 Frontend Implementation 27
4.8 PDF Report Generation 28
4.9 Security and Privacy 28
4.10 Data Protection and Ethical Use 28
4.11 Testing 29
4.12 Deployment 31
4.13 Project Timeline 31
Chapter 5 Results and Discussion 33
5.1 Experimental Setup 33
5.2 Results 33
5.2.1 Image Detection: Seen Versus Unseen Sources 33
5.2.2 Score Distributions, Threshold Sensitivity and Reliability 36
5.2.3 Face-Swap Check in the Image Pipeline 38
5.2.4 Video Detection 38
5.2.5 Audio Detection 40
5.2.6 Robustness to Everyday Degradation 41
5.2.7 Explanations, Provenance and Calibration 43
5.2.8 Fairness Across Demographic Groups 44
5.2.9 Identity Verification 44
5.2.10 The Working System 45
5.3 Comparison with Existing Methods 52
5.4 Discussion and Limitations 53
Chapter 6 Conclusion and Future Scope 55
6.1 Conclusion 55
6.2 Future Scope 55
Chapter 7 List of Publications 57
7.1 List of Published Papers 57
7.2 Details of Participation in Project Competitions 57
References 58
Appendix A: Installation and User Guide 62
Appendix B: Selected Source Code 65
Appendix C: Additional Screenshots and Sample Report 68

List of Figures

Figure 3.1: Block diagram of the proposed system 10
Figure 3.2: Image detection pipeline 13
Figure 3.3: Video detection pipeline 14
Figure 3.4: Audio detection pipeline 15
Figure 3.5: Verdict bands around the decision threshold 16
Figure 3.6: Provenance checks and how they reach the user 17
Figure 3.7: ProctorLens examination flow 18
Figure 3.8: Signals computed for every monitoring check 19
Figure 4.1: Use-case diagram 23
Figure 4.2: Data flow diagram, level 0 24
Figure 4.3: Data flow diagram, level 1 24
Figure 4.4: Sequence diagram: image or audio scan and evidence requests 25
Figure 4.5: Sequence diagram: background video scan 25
Figure 4.6: Entity-relationship diagram 26
Figure 4.7: Production deployment 31
Figure 4.8: Implementation timeline 32
Figure 5.1: Image detection by training status of the source (threshold 0.5) 34
Figure 5.2: Confusion matrices of the image ensemble (6,532 files) and the epoch-11 video model (310 clips) 35
Figure 5.3: Share of AI images caught per unseen source (deployed max ensemble) 35
Figure 5.4: ROC and precision-recall curves of the image sub-models and ensembles (6,532 images) 36
Figure 5.5: Distribution of the deployed image ensemble's fake probability by true class (6,532 images) 36
Figure 5.6: Balanced accuracy of each image configuration against the decision threshold 37
Figure 5.7: Reliability diagram of the raw image ensemble score 37
Figure 5.8: Face-swap check versus whole-image models (ROC-AUC) 38
Figure 5.9: Video preprocessing variants on 300 unseen RTFS clips 39

Figure 5.10: ROC and precision-recall curves of Video Model v1 (epoch 11) on 310 unseen clips 39
Figure 5.11: Video ROC-AUC against the short side of the video 40
Figure 5.12: Video Model v1: epoch 11 versus round 4 on held-out videos 40
Figure 5.13: Audio model results on held-out acceptance sets 41
Figure 5.14: Image robustness on seen sources: specificity and recall under degradation 41
Figure 5.15: Deployed image ensemble under all 21 robustness settings (AUC, fake recall and real specificity, seen and unseen sources) 42
Figure 5.16: Video robustness with every sampled frame degraded (30 RTFS clips) 43
Figure 5.17: Expected calibration error before and after calibration 44
Figure 5.18: Real-photo specificity per gender and race group (600 UTKFace portraits) 44
Figure 5.19: Distribution of cosine similarity for same-person and different-person LFW pairs 45
Figure 5.20: Identity matching at threshold 0.44 on LFW 45
Figure 5.21: Home page 46
Figure 5.22: Verify page: upload of an image, video or audio file 46
Figure 5.23: Scan history (owner-only) 47
Figure 5.24: Dashboard built from real counts only 47
Figure 5.25: Result for a ChatGPT-generated photograph: REAL by the models, AI-declared by provenance 48
Figure 5.26: Result for a portrait-app image: UNCERTAIN by the models, AI-declared by metadata 49
Figure 5.27: Video result: per-frame scores, face crops and Grad-CAM on the most suspicious frames 50
Figure 5.28: Audio result: per-window score and input-gradient saliency over the spectrogram 51
Figure 5.29: First page of the generated PDF forensic report for the video 51
Figure 5.30: ProctorLens student hub: joining an organisation to see its exams 52
Figure C.1: Login page 68
Figure C.2: Result card for a genuine photograph: Authentic, no provenance metadata, explanation collapsed 68
Figure C.3: Generated PDF report for the ChatGPT photograph, page 1: verdict and technical details 69
Figure C.4: Generated PDF report for the ChatGPT photograph, page 2: explanation and provenance conflict note 70
Figure C.5: Generated PDF report for the face-swap video, page 2: per-frame explanation 71

List of Tables

Table 2.1: Summary of literature survey 7
Table 3.1: Dataset description: training sources 10
Table 3.2: Dataset description: evaluation sources 11
Table 4.1: Hardware and software requirements 22
Table 4.2: Main REST endpoints 27
Table 4.3: Representative test cases 30
Table 5.1: Image detection by training status of the source 34
Table 5.2: Performance comparison of the evaluated methods 52
Table 5.3: Capability comparison with typical existing approaches 53

Abbreviations

AASIST – Audio Anti-Spoofing using Integrated Spectro-Temporal graph attention
AI – Artificial Intelligence
API – Application Programming Interface
AUC – Area Under the (ROC) Curve
C2PA – Coalition for Content Provenance and Authenticity
CI – Continuous Integration
CLIP – Contrastive Language-Image Pre-training
CNN – Convolutional Neural Network
CSP – Content-Security-Policy
DFD – Data Flow Diagram
DWT – Discrete Wavelet Transform
ECE – Expected Calibration Error
ELA – Error Level Analysis
EXIF – Exchangeable Image File Format
FF++ – FaceForensics++
Grad-CAM – Gradient-weighted Class Activation Mapping
GAN – Generative Adversarial Network
JWT – JSON Web Token
LFW – Labeled Faces in the Wild
MCQ – Multiple-Choice Question
MTCNN – Multi-task Cascaded Convolutional Networks
ONNX – Open Neural Network Exchange
PDF – Portable Document Format
RMS – Root Mean Square
ROC – Receiver Operating Characteristic
RTFS – RealTimeFaceSwap-10k dataset
SD – Stable Diffusion
SPA – Single-Page Application
VAD – Voice Activity Detection
ViT – Vision Transformer
XMP – Extensible Metadata Platform
YOLO – You Only Look Once (object detector)

Chapter 1
Introduction

Until a few years ago, producing a convincing fake photograph took skill and hours of editing. Today it takes a one-line text prompt. Diffusion models render photorealistic scenes on a laptop, consumer face-swap apps replace a face in a phone video within minutes, and voice-cloning services need only a short sample of the target speaker. Journalists, fact-checkers, teachers, recruiters and ordinary users are increasingly asked to decide whether a file is genuine, usually without any forensic training and without access to the tools researchers use.

TruthLens is a web platform built to help with that decision. A user uploads an image, a video or an audio clip; the system routes it to a detector trained for that kind of media and returns a verdict of REAL, UNCERTAIN or FAKE with a confidence score. Unlike a bare classifier, every verdict is accompanied by the evidence behind it: heat-maps of the regions the model relied on, per-frame or per-window scores, the file's own provenance metadata, and a downloadable PDF forensic report. The same face-analysis stack is reused by **ProctorLens**, a module that verifies a candidate's identity against a government ID and monitors a proctored online examination.

This report documents the complete system as submitted for the final review: its design, the three detection pipelines and how each one parses its input, the evidence layer, the proctoring module, the implementation and deployment, and a measured evaluation that deliberately reports where the system fails as well as where it works.

1.1 Background

Synthetic media is produced by three broad families of generative models. **Generative adversarial networks** (GANs) such as StyleGAN produce faces and objects by training a generator against a discriminator. **Diffusion models** such as Stable Diffusion [1], Midjourney and DALL-E iteratively denoise random noise into an image conditioned on a text prompt, and now dominate image generation. **Face-swap and re-enactment systems**, from the autoencoder methods in FaceForensics++ [2] to modern one-shot

swappers such as inswapper, transplant one person's face onto another's head in every frame of a video. For audio, text-to-speech and voice-conversion systems clone a speaker's voice from seconds of reference audio [3].

Detection research has grown alongside, but it is split by modality, each with its own benchmarks: FaceForensics++ [2] and Celeb-DF [4] for face video, CIFAKE [5] and GenImage [6] for images, and ASVspoof [3] for speech. Most detectors are deep convolutional or transformer networks trained as binary real-versus-fake classifiers. A complementary line of work, **content provenance**, attaches cryptographically signed records of how a file was made: the C2PA standard [7] lets cameras and generators sign a manifest that travels with the file, and OpenAI, Adobe and others now embed such credentials in their outputs.

1.2 Motivation

Three observations motivated the project. First, deepfakes have moved from novelty to everyday harm: fake evidence in disputes, non-consensual imagery, voice-clone fraud calls and impersonation in online interviews and examinations. Second, the free detectors available to the public usually return a single number with no explanation, so the user cannot judge whether to trust it, and they rarely say how they were evaluated. Third, published detectors are known to transfer poorly to generators they were not trained on [8, 9, 6], so headline benchmark accuracy overstates what a user will actually experience.

We therefore set out to build a system that is useful to a non-expert *and* honest about its limits: one that covers all three media types, explains each decision, uses provenance where it exists, refuses to force a borderline call, and is evaluated in a way that keeps familiar data apart from unfamiliar data. Online examinations, where a candidate may be impersonated or replaced by a live deepfake, are a natural second application of the same face-analysis components and became the ProctorLens module.

1.3 Problem Statement

Given an image, video or audio file supplied by a user, determine whether it is likely to have been generated or manipulated by AI, and present that judgement together with inspectable evidence and a calibrated statement of confidence. The system must handle borderline cases explicitly, must not treat missing metadata as proof of authenticity, must keep every user's data private, and its accuracy must be measured separately on data

similar to and different from its training data. In addition, for proctored online examinations, verify that the candidate matches a government-issued ID and continuously flag behaviour that suggests impersonation or malpractice.

1.4 Objectives

The objectives of this project are:

- To study existing techniques for detecting AI-generated images, face-swap videos and synthetic speech, together with explanation, provenance and calibration methods.
- To design and implement a multimodal detection platform with a dedicated, trained or fine-tuned detector for each media type and a common verdict, report and history layer.
- To add an evidence layer: Grad-CAM and saliency explanations, C2PA and metadata provenance, invisible-watermark detection and calibrated confidence, without letting missing provenance count as authenticity.
- To build ProctorLens: government-ID verification, live identity matching and continuous monitoring of proctored multiple-choice examinations with server-side enforcement.
- To evaluate the deployed models separately on seen and unseen data sources, and to measure robustness to everyday degradation, explanation faithfulness, provenance survival, calibration and error rates across demographic groups.
- To harden the system for deployment: authentication, owner-only access, upload validation, rate limits, automated tests, CI and containerised production configuration.

1.5 Scope of the Work

In scope. Still images (JPEG, PNG, WebP, HEIC/HEIF), videos (MP4, MOV, WebM and similar containers) and audio (WAV, MP3, M4A, OGG, Opus, WebM) up to 100 MB; binary classification into real or AI-generated/manipulated with a three-way banded verdict; per-modality explanations; provenance checks; PDF reports; per-user history, dashboard and feedback; identity verification and proctoring of MCQ examinations created by an organisation; Docker-based deployment on a single CPU server.

Out of scope and known limits. The video model targets face-swap and lip-sync deepfakes; fully generated video (for example Sora or Kling) is outside what it was trained on. Detection generalises poorly to image generators absent from the training data. The system produces leads and evidence for human review, not proof, and is not intended for legal, employment or disciplinary decisions without a human in the loop. Inference runs on CPU in a single server process; horizontal scaling would need a shared job store.

1.6 Organization of the Report

The rest of the report is organized as follows. Chapter 2 reviews the related literature and identifies the research gaps. Chapter 3 presents the proposed methodology: the architecture, the datasets, and each pipeline (image, video, audio, evidence layer and ProctorLens) step by step. Chapter 4 describes the system design and implementation: requirements, UML and data-flow diagrams, database schema, API, frontend, security, testing and deployment. Chapter 5 discusses the experimental results with screenshots of the working system. Chapter 6 concludes the report and outlines future work, and Chapter 7 lists the publication arising from the project.

Chapter 2
Literature Survey

This chapter reviews recent work on the three detection problems and on the supporting techniques the project uses. For each work we note the method, the data, the reported result and the limitation that matters for a deployed system.

2.1 Review of Existing Work

2.1.1 AI-Generated Image Detection

Wang et al. [8] trained a ResNet-50 on images from a single GAN (ProGAN) with blur and JPEG augmentation and showed that it transfers surprisingly well to other GANs of that period, establishing augmentation as a key ingredient. Frank et al. [10] showed that the up-sampling layers of GANs leave regular artefacts in the frequency spectrum that a simple classifier can exploit, although such cues are fragile to re-compression. With diffusion models the picture changed: the GenImage benchmark [6] (eight generators, over a million images) reports large accuracy drops when a detector is tested on generators it was not trained on. Ojha et al. [9] proposed classifying frozen CLIP [11] features with a light head; because the feature space is not shaped around one generator, it generalises markedly better than end-to-end training. Bird and Lotfi [5] released CIFAKE, a balanced 120,000-image dataset of CIFAR-10 photographs and Stable Diffusion 1.4 images at 32×32 pixels, and reached about 93% accuracy with a CNN, but the images are far smaller than real uploads. ConvNeXt [12] modernised the ResNet design with transformer-style choices and is a strong, efficient backbone for fine-tuning.

2.1.2 Face-Manipulation Video Detection

Rössler et al. [2] introduced FaceForensics++, with four manipulation methods (Deepfakes, Face2Face, FaceSwap, NeuralTextures) and an XceptionNet baseline on face crops that is highly accurate on high-quality in-distribution data but degrades under compression and on unseen methods. Celeb-DF [4] provided higher-quality celebrity swaps on which earlier detectors' AUC dropped sharply. The DFDC dataset [13] added scale and diversity, and its strongest entries were ensembles of EfficientNet [14] models

applied to face crops. WildDeepfake [15] collected deepfakes from the internet and showed a further gap between laboratory and real-world performance. A recurring weakness of the face-crop design is the face detector itself: if it locks onto the wrong region, the classifier never sees the manipulation.

2.1.3 Synthetic Speech Detection

The ASVspoof 2019 challenge [3] established logical-access benchmarks of text-to-speech and voice-conversion attacks. Jung et al. [16] proposed AASIST, which models spectral and temporal sub-graphs with graph attention directly from raw waveforms and achieves very low equal-error rates on ASVspoof 2019 LA. Tak et al. [17] showed that a self-supervised wav2vec 2.0 front-end [18], fine-tuned with data augmentation, improves generalisation considerably. Müller et al. [19] collected the In-the-Wild dataset of real public recordings and found that models that excel on ASVspoof degrade sharply on real-world audio, which motivates training on diverse sources rather than one benchmark.

2.1.4 Explanation, Provenance and Calibration

Grad-CAM [20] weights the last convolutional feature maps by the gradient of the target class to produce a coarse localisation of the evidence a CNN used; input-gradient saliency [21] is a simpler alternative for models without a convolutional map. Neither proves manipulation; both show what the score depends on. Provenance answers a different question: the C2PA specification [7] defines signed manifests that record a file's origin and edits, and Stable Diffusion's reference pipeline embeds a fixed invisible watermark [1, 22]. Error level analysis [23] visualises JPEG re-compression differences but is easily misread. Guo et al. [24] showed that modern networks are over-confident and that temperature scaling is a simple, effective fix; Platt scaling [25] fits a logistic map from scores to probabilities.

2.1.5 Face Verification and Online Proctoring

MTCNN [26] jointly detects faces and five facial landmarks with a cascade of three small CNNs. FaceNet [27] maps a face to a compact embedding in which Euclidean or cosine distance measures identity, and InceptionResNet-v1 models trained on VGGFace2 [28] are widely used for verification; LFW [29] remains the standard verification benchmark. Single-stage detectors of the YOLO family [30] detect everyday objects such as phones and books in real time. Commercial proctoring tools combine such components but rarely publish thresholds or error rates, and automated flags are known to raise fairness and

privacy concerns, which argues for transparent, reviewable signals rather than automatic verdicts.

2.2 Summary of Literature

Table 2.1 summarizes the literature reviewed.

Table 2.1: Summary of literature survey

Sr. No.	Author(s) & Year	Method / Technique	Dataset	Result	Limitation
1	Rössler et al., 2019 [2]	XceptionNet on face crops	FaceForensics++	High accuracy on HQ in-distribution fakes	Drops under compression and unseen methods
2	Li et al., 2020 [4]	Benchmark of improved face swaps	Celeb-DF v2	Existing detectors' AUC falls sharply	Dataset only; celebrity faces
3	Wang et al., 2020 [8]	ResNet-50 + blur/JPEG augmentation	ProGAN, 11 GANs	Transfers across GANs	Weaker on diffusion models
4	Frank et al., 2020 [10]	Frequency (DCT) spectrum cues	GAN images	Up-sampling artefacts are detectable	Fragile to re-compression
5	Jung et al., 2022 [16]	AASIST graph attention on raw audio	ASVspoof 2019 LA	Very low EER on benchmark	Degrades on real-world audio
6	Tak et al., 2022 [17]	wav2vec 2.0 front-end + augmentation	ASVspoof 2021 LA/DF	Better generalisation	Heavy model, benchmark-bound
7	Müller et al., 2022 [19]	Evaluation of 12 audio detectors	In-the-Wild	Large drop on real recordings	Exposes, does not solve, the gap
8	Ojha et al., 2023 [9]	Frozen CLIP ViT + linear/NN head	GAN + diffusion	Better unseen-generator accuracy	Still misses some generators
9	Zhu et al., 2023 [6]	Million-image benchmark	GenImage (8 generators)	Cross-generator accuracy drops	Format biases (JPEG vs PNG)
10	Bird & Lotfi, 2024 [5]	CNN on 32×32 images	CIFAKE	About 93% accuracy	Tiny images, one generator

2.3 Research Gaps

The review points to the following gaps, which this project addresses:

- **Single-modality tools.** Published detectors cover one media type; a user facing an image, a video and a voice note needs three separate tools with different outputs.
- **Bare scores.** Most tools return a probability with no explanation, no statement of uncertainty and no indication of how reliable the number is.

- **Seen-versus-unseen reporting.** Accuracy is usually reported on held-out parts of the training benchmark, which measures fit rather than generalisation to new generators.
- **Provenance in isolation.** C2PA credentials and watermarks are rarely combined with detector scores, and the question of what to show when the two disagree is seldom addressed.
- **Robustness and calibration.** Behaviour under ordinary sharing (re-compression, resizing, messaging apps) and the calibration of confidence are rarely measured for a deployed system.
- **Opaque proctoring.** Proctoring products seldom publish their identity thresholds or flag logic, and do not check whether the face on camera is itself synthetic.

Chapter 3

Proposed Methodology

TruthLens follows one principle throughout: a detector score is one strand of evidence, not the final answer. Each upload is routed to a modality-specific detector, the score is mapped to a three-way verdict, and an independent evidence layer adds explanations and provenance that the user can inspect. This chapter describes the architecture, the data, and each pipeline in the order in which it processes a file.

3.1 System Architecture

The block diagram of the proposed system is shown in Figure 3.1. A React single-page application talks to a FastAPI backend through an nginx reverse proxy. The backend is organised in layers: routers (authentication, upload, detection, background jobs, reports, dashboard, feedback, identity, exams and administration) validate requests and enforce ownership; detectors and ProctorLens services run the models; the evidence layer computes explanations, provenance and calibration on demand; and the storage layer keeps results in SQLite through async SQLAlchemy and files on disk. Models are loaded once per process and shared between detectors and explainers.

Two design decisions are visible in the figure. First, the evidence layer is separate from the detectors: an explanation or provenance failure can never change or break a stored verdict. Second, slow work (video scans and proctoring checks) runs in background job managers with a bounded thread pool, so a long video never freezes the server for other users.

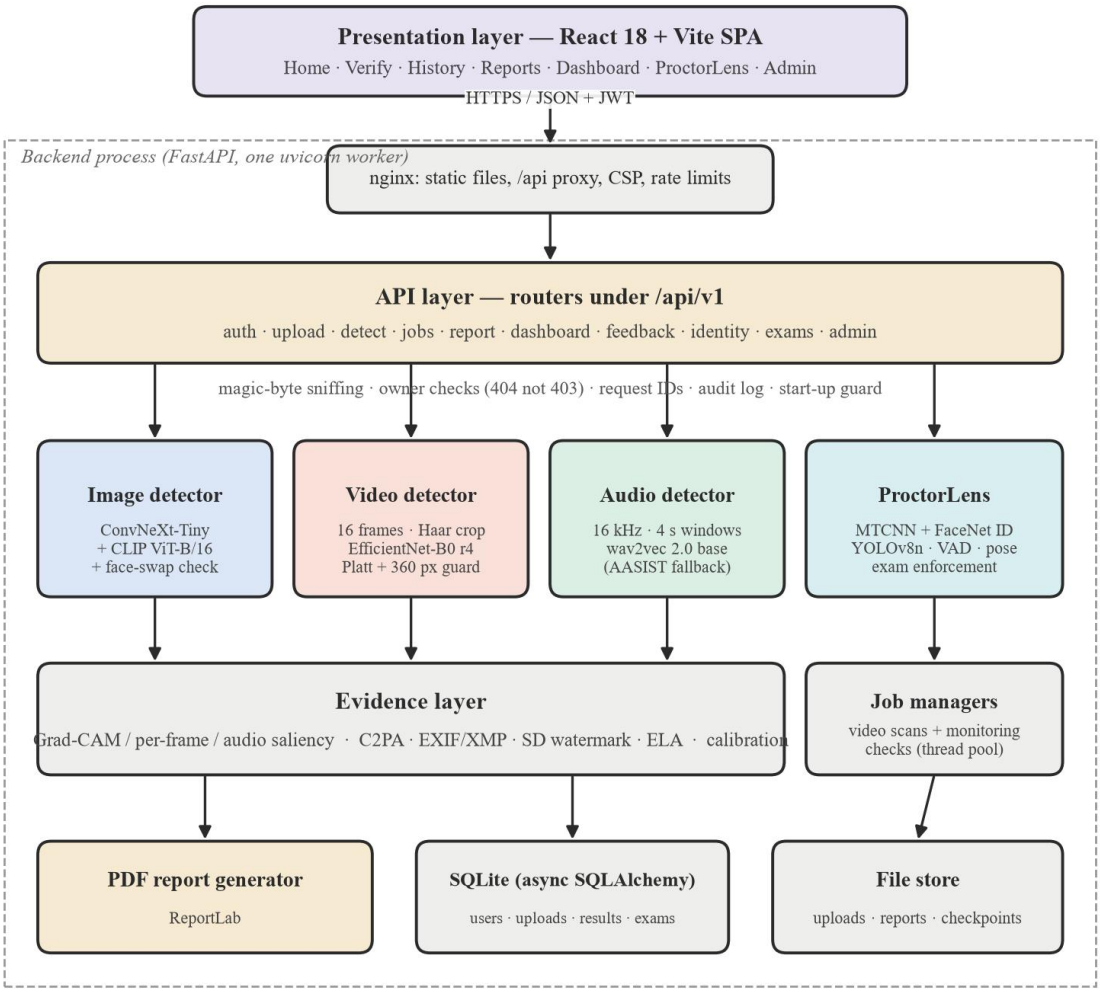


Figure 3.1: Block diagram of the proposed system

3.2 Dataset Description

The details of the datasets used are given in Table 3.1 (training) and Table 3.2 (evaluation). A source is called *seen* when the model was trained on data from it, so accuracy on it measures fit; *unseen* sources were never used for training and measure generalisation. The datasets themselves are not stored in the repository; only small test samples are.

Table 3.1: Dataset description: training sources

Parameter	Image detector	Video detector	Audio detector
Name of dataset	CIFAKE; AI-vs-Human-Generated; 140k Real and Fake Faces; DeepDetect-2025; DiffusionDB portrait subset	FaceForensics++, Celeb-DF, DFDC, DFD, RTFS-10k, FakeAVCeleb, phone-recorded swaps (round 4)	ASVspoof 2019 (VC attacks), DEEP-VOICE, In-the-Wild, LibriSpeech, VCC 2020

Parameter	Image detector	Video detector	Audio detector
Source	Kaggle; Hugging Face (DiffusionDB, CC0)	Research releases, Kaggle, Hugging Face, team recordings	Challenge releases, Kaggle, OpenSLR
Number of samples	120,000 + ~80,000 + 140,000 + 100,000+ images; 257 hand-reviewed portrait-app fakes (CLIP only)	Up to 600 crops per class per dataset per epoch; 8–16 frames per video	Clip-level, split by speaker or by conversion system
Number of classes	2 (real, AI-generated)	2 (real, face-swap)	2 (genuine, deepfake)
Train / Test split	Dataset test folders; exact hold-out split not recorded	Grouped by source clip and person; 926-video validation split	Speaker-split / system-split held-out sets

Table 3.2: Dataset description: evaluation sources

Source	Files	Status	Used for
CIFAKE, AI-vs-Human, 140k Faces, DeepDetect (test folders)	4,000	Seen	Image fit
GenImage subset: real / SD / Midjourney / BigGAN	600 / 300 / 300 / 300	Unseen	Image generalisation
OpenFake (2025 generators): real / fake	500 / 500	Unseen	Image generalisation
ChatGPT "everyday" photos; team photos; portrait-app images	8; 9; 15	Unseen / CLIP only	Blind spots
RTFS-10k: real / inswapper / uniface	150 / 75 / 75	Unseen (epoch 11)	Video
356 local held-out videos (round 4 comparison)	185 real / 171 fake	Held out	Video round 4
UTKFace real portraits, 10 gender × race groups	600	Unseen	Fairness
LFW DevTest pairs (same / different person)	500 / 500	Unseen	Identity threshold
RWFS real-app face swaps + real photos	500 / 500	Unseen (r1)	Face-swap check

Several dataset quirks affect how results must be read: a trivial JPEG statistic separates DeepDetect's real and fake images with AUC 0.995 (a compression-history shortcut); in GenImage the real images are JPEG and the fakes PNG; CIFAKE is natively 32×32; and RTFS real and fake clips went through different encodes.

3.3 Algorithm / Model Description

The workflow common to all media types is:

1. The user uploads a file; the server checks its size and its leading bytes (magic-byte sniffing) against the declared media type, stores it and creates an upload record owned by that user.
2. Detection is dispatched by media type. Images and audio are scored within the request; videos run as a background job that reports real progress.
3. The detector returns a fake probability p, which is mapped to a REAL / UNCERTAIN / FAKE verdict, and a calibrated confidence is computed.
4. On demand, the evidence layer computes the explanation and the provenance signals; if provenance declares AI generation while the models do not, a conflict note is attached.
5. The verdict, sub-scores, evidence and caveats are shown in the interface and rendered into a PDF report; the user may give feedback.

3.3.1 Image Pipeline

Figure 3.2 shows how an image is parsed. After decoding with Pillow (HEIC/HEIF through pillow-heif), three scorers run in parallel on the same file:

- **ConvNeXt-Tiny** [12], fine-tuned by the team. It was first trained on CIFAKE, then fine-tuned on four sources (CIFAKE, AI-vs-Human, 140k Faces and DeepDetect-2025, which covers StyleGAN3, DALL-E 3, Midjourney and SD3) with random JPEG re-compression, blur and resize augmentation. The image is resized directly to 224×224 with bicubic interpolation and ImageNet normalisation. An earlier variant first shrank every image to 32×32 to match CIFAKE; on a held-out evaluation of 3,000 images per source it cut face-image accuracy from 99.8% to 57%, so it was removed.
- **CLIP ViT-B/16** [11], with the backbone frozen and only a small MLP head trained (512 → 256 → 2, dropout 0.3), following [9]. Round 3 of the head added 257 hand-reviewed Stable Diffusion portraits retrieved from DiffusionDB [31] by prompt text, after both sub-models were found to call stylised "portrait-app" images REAL with over 99.9% confidence. CLIP's accuracy on that genre rose from 26.7% to 93.3%.

- **Face-swap check.** Whole-image models are near chance on face swaps, because a swap is a real photograph with only one face replaced. If a frontal face at least 64 px wide is found (Haar cascade first, MTCNN as fallback), the face crop is scored by an EfficientNet-B0 photo model (round 2: a 50/50 weight blend of the video model and a fine-tune on real-app swaps) and Platt-calibrated. Any failure here means only "check not applied"; the scan still succeeds.

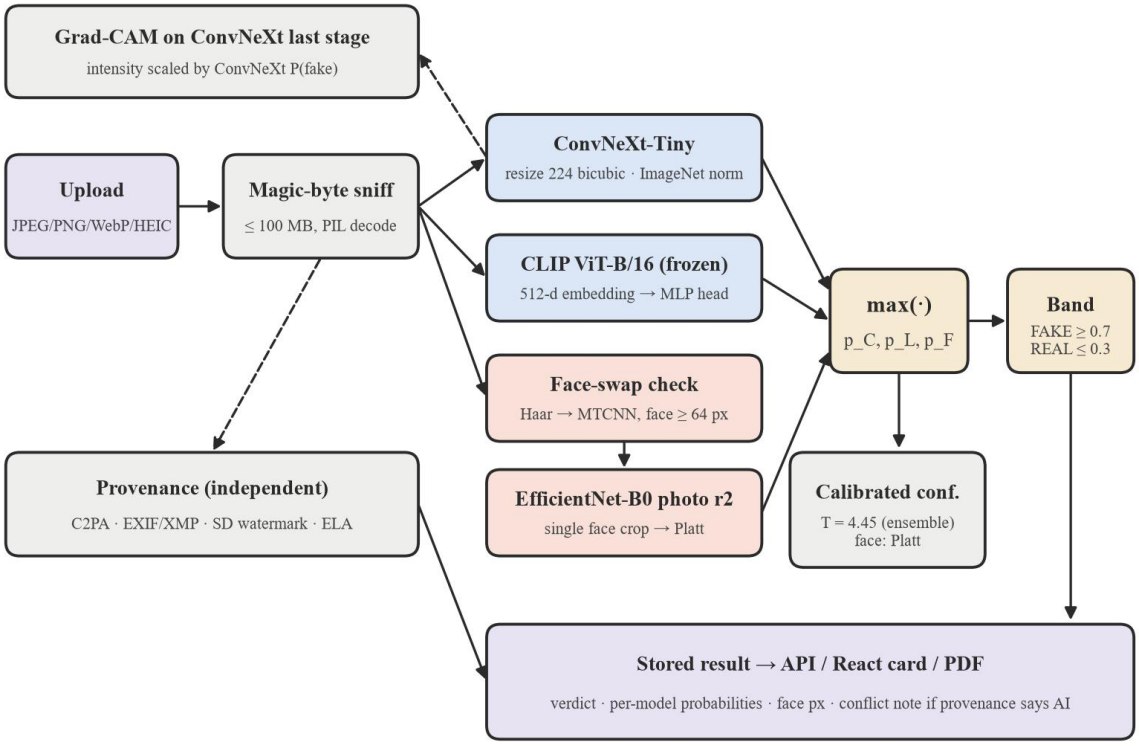


Figure 3.2: Image detection pipeline

With p_C, p_L and p_F the fake probabilities of ConvNeXt, CLIP and the face check, the image score is given by Equation (3.1):

$$p_{img} = max(p_C, p_L, p_F) \tag{3.1}$$

The maximum rather than the mean was chosen deliberately. ConvNeXt was not retrained on the portrait genre and remains blind to it (6.7% caught); averaging diluted CLIP's 93.3% on that genre to 66.7%. The maximum keeps either model's detection, at a measured cost in false alarms on ordinary photographs (Chapter 5).

3.3.2 Video Pipeline

Figure 3.3 shows how a video is parsed. The pipeline was verified line by line against the original Celeb-DF v2 evaluation script of the model, and a regression test guards that parity.

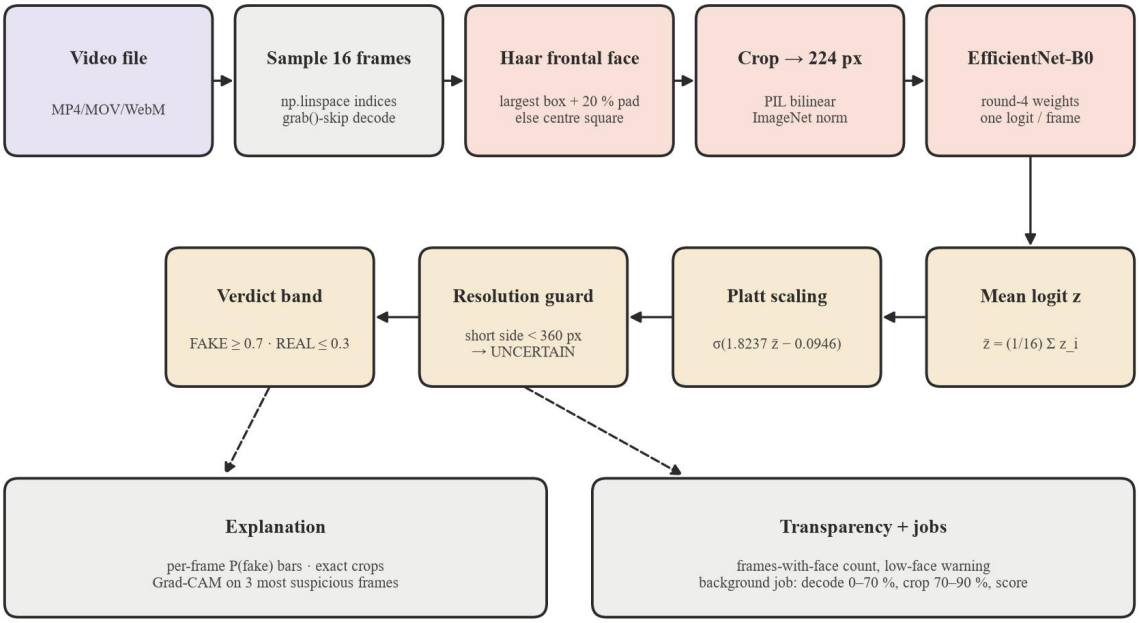


Figure 3.3: Video detection pipeline

1. Frame sampling: sixteen frame indices are chosen with numpy linspace over the whole clip and decoded with OpenCV, skipping unneeded frames with grab(). Clips shorter than sixteen frames are rejected with a clear error.
2. Face crop: in each frame a Haar cascade [32] finds the largest frontal face, the box is enlarged by 20%, and the crop is resized to 224×224 with PIL bilinear interpolation and ImageNet normalisation. If no face is found, a centre square is used; the system records how many of the sixteen frames contained a detected face and warns when fewer than half did.
3. Scoring: an EfficientNet-B0 [14] (Video Model v1, round 4) outputs one fake logit z_i per frame in one batched forward pass; the clip logit is their mean.
4. Calibration: the mean logit is mapped through Platt scaling, Equation (3.2), fitted on the round-4 validation split (926 videos, a = 1.8237, b = −0.0946).
5. Resolution guard: if the short side of the video is under 360 px the verdict is withheld as UNCERTAIN, because accuracy collapses on small faces (Chapter 5).
6. Verdict: the calibrated probability is banded as in Equation (3.3).

$$\hat{p} = \sigma(a \cdot \bar{z} + b), \quad \bar{z} = \frac{1}{16}\sum_{i=1}^{16} z_i \tag{3.2}$$

Round 4 was fine-tuned from the original epoch-11 checkpoint on Kaggle with production-identical face crops, real-world degradation augmentation (JPEG re-compression, down-scaling, blur, noise), four epochs, a learning rate of 4×10^{-5}, batch size 64, eight frames per video (sixteen for the small phone sources) and at most 600 crops per class per dataset per epoch so that no single source dominates; the best epoch was selected by validation AUC.

3.3.3 Audio Pipeline

Figure 3.4 shows how audio is parsed. Speech is decoded with PyAV to 16 kHz mono and RMS-normalised once over the whole clip to −20 dBFS, exactly as in training. Up to six 4-second windows (64,000 samples) are taken at evenly spaced offsets; clips shorter than one window are tiled to length.

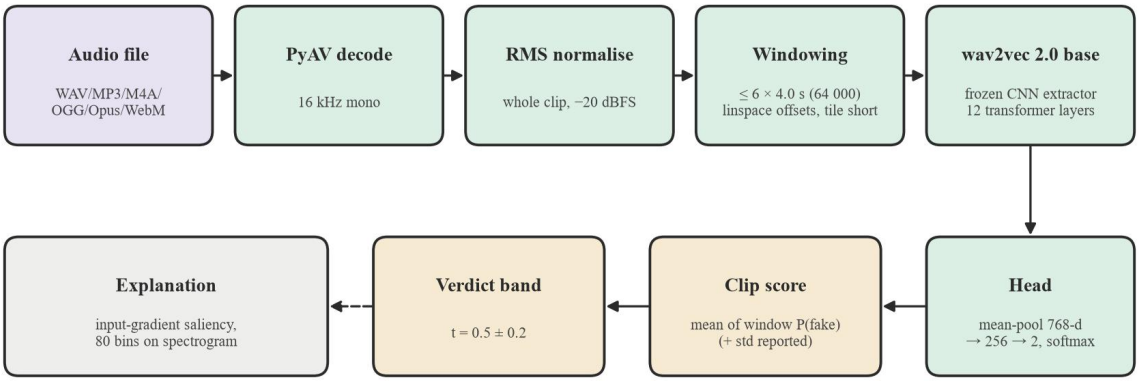


Figure 3.4: Audio detection pipeline

Each window is passed through a wav2vec 2.0 base model [18] whose convolutional feature extractor is frozen; the 768-dimensional hidden states are mean-pooled and a small head (768 → 256 → 2) gives the deepfake probability by softmax. The clip score is the mean over windows and the standard deviation is reported alongside. The model was fine-tuned in successive rounds (v2 to v9) on voice-conversion attacks from ASVspoof 2019 [3], DEEP-VOICE [33], In-the-Wild [19] split by speaker, genuine LibriSpeech [34] narration split by speaker (added to remove false alarms on clean studio audio) and VCC 2020 [35] voice-conversion systems split by system. Round 9 used AdamW, a learning rate of 10^{-5}, three epochs, batch size 8 and class-weighted cross-entropy. The pretrained AASIST [16] ONNX model remains selectable as a fallback.

3.3.4 Verdict Bands and Calibrated Confidence

All detectors map their score p to a three-way verdict around a threshold t with a margin of 0.2 (Figure 3.5). For images and audio t = 0.5 is applied to the raw probability; for video it is applied to the Platt-calibrated probability:

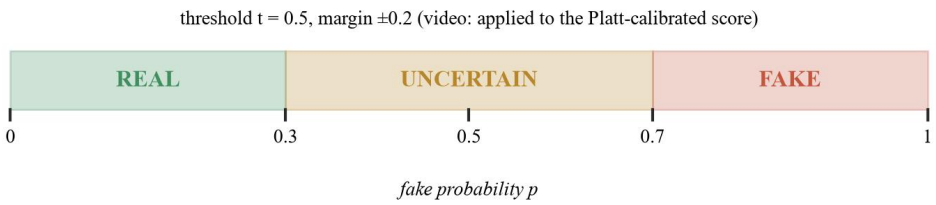


Figure 3.5: Verdict bands around the decision threshold

$$v(p) = FAKE\ if\ p \geq t + 0.2;\quad REAL\ if\ p \leq t - 0.2;\quad UNCERTAIN\ otherwise \tag{3.3}$$

The UNCERTAIN band reflects a deliberate choice that a borderline score is reported as borderline instead of being forced into a binary answer. Confidence is stored as the probability of the verdict given, max(p, 1−p), and a separately calibrated confidence is shown next to it. For images it uses temperature scaling [24], Equation (3.4), with T = 4.45 fitted on 6,500 labelled evaluation scores; no audio calibration is shown because no labelled audio evaluation set exists in the harness yet, and the interface says "not available" rather than guessing.

$$\hat{p} = \sigma\left(\frac{logit(p)}{T}\right),\quad \sigma(x) = \frac{1}{1+e^{-x}} \tag{3.4}$$

3.3.5 Explanation Layer

Explanations are recomputed on demand with the deployed models and verified to reproduce the production numbers exactly. For images, Grad-CAM [20] is computed on ConvNeXt's last stage with respect to the fake logit. Because a normalised Grad-CAM map always shows some bright region, even on images the model considers real, the map's intensity is scaled by ConvNeXt's own fake probability, so a confident REAL verdict shows a near-empty map. CLIP and the face check have no spatial map; when either drives the verdict, the interface says so instead of presenting ConvNeXt's map as the explanation. For video, the user sees each frame's probability, the exact crops the model received and Grad-CAM on the three most suspicious frames. For audio, Grad-CAM does not apply because wav2vec 2.0 has no late two-dimensional feature map, so input-gradient saliency [21] of the deepfake probability with respect to the waveform is pooled into 80 time bins and overlaid on the spectrogram of the most suspicious window.

3.3.6 Provenance Layer

The provenance service (Figure 3.6) runs independently of the detectors and reports four kinds of evidence:

- **C2PA Content Credentials** [7], validated with the official SDK against the published trust list: whether the signature is valid, whether the file is unchanged since signing, whether the signer is trusted, and whether the manifest declares AI generation through the IPTC digital-source-type field.
- **Embedded metadata**: camera EXIF, editing software, the presence (never the coordinates) of GPS data, XMP AI source type, and generation parameters (prompt, seed, sampler) left by Stable Diffusion front-ends such as AUTOMATIC1111 and ComfyUI.
- **Stable Diffusion / SDXL invisible watermark** [22]: the fixed 48-bit message embedded by the reference pipeline is decoded with a re-implementation (Haar DWT in PyWavelets) verified bit-for-bit against the reference library; a match requires all 48 bits, and for video eight sampled frames are checked.
- **Error level analysis** for JPEG files, shown as a visual aid only and never scored.

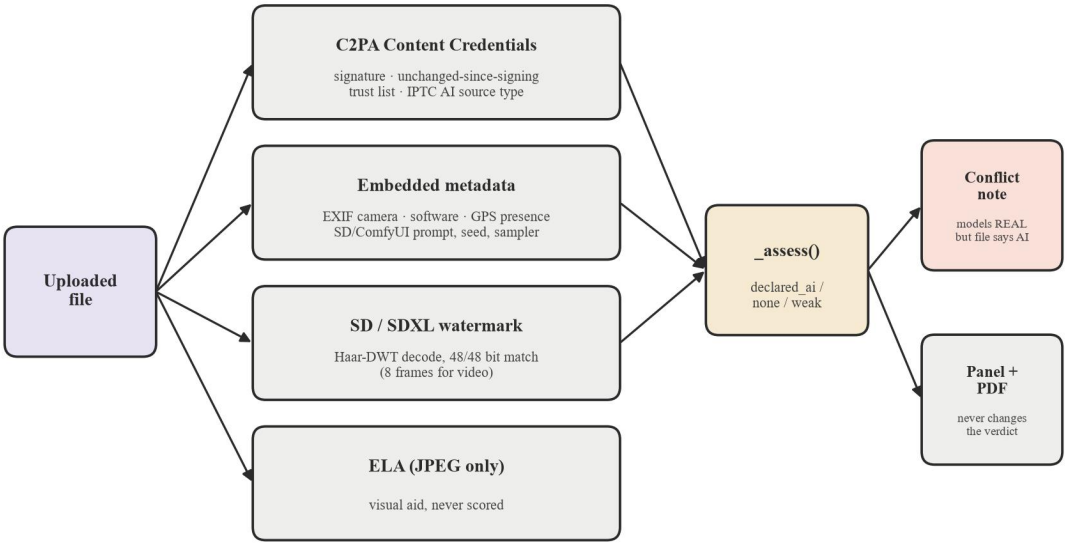


Figure 3.6: Provenance checks and how they reach the user

Provenance never changes the stored verdict, because metadata can be removed by any re-save. A present AI declaration is strong evidence; an absent one says nothing either way. When the models say REAL or UNCERTAIN but the file declares itself AI-generated, the API, the interface and the PDF all show an explicit conflict note, and the interface switches the headline to the warning style.

3.3.7 ProctorLens: Identity Verification and Exam Monitoring

ProctorLens reuses the face-analysis stack for proctored online examinations (Figure 3.7). An organisation account creates MCQ exams and shares a private join code; a student links to the organisation, opens an exam, and must pass a fresh identity check for every attempt: a government-ID photo is checked for exactly one sharp face, the photo itself is deleted as soon as its FaceNet [27] embedding has been extracted (also when the check fails), and a live selfie must match that embedding.

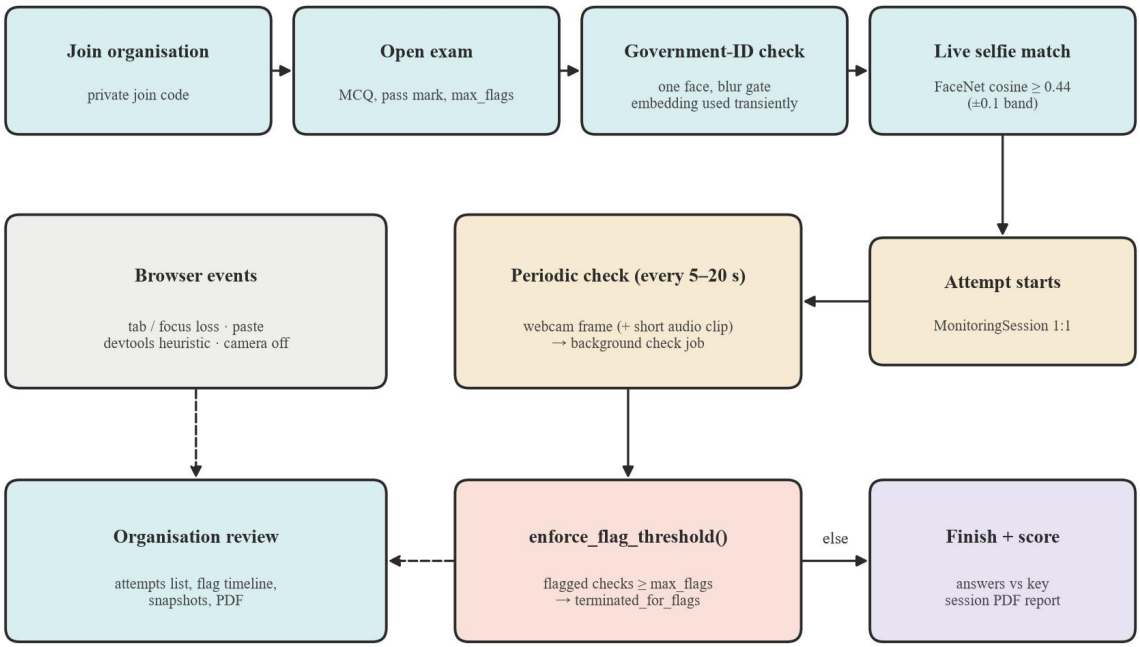


Figure 3.7: ProctorLens examination flow

Identity is decided by the cosine similarity of 512-dimensional InceptionResNet-v1 (VGGFace2) embeddings of MTCNN [26] aligned faces, Equation (3.5), banded around a threshold of 0.44 with a margin of 0.1. The threshold was calibrated on 1,000 labelled LFW [29] pairs as the accuracy-maximising point.

$$s(u,v) = \frac{u \cdot v}{\|u\|\,\|v\|},\ \ match\ if\ s \geq 0.44 + 0.1,\ \ mismatch\ if\ s \leq 0.44 - 0.1 \quad (3.5)$$

During the attempt the browser sends a webcam frame (and optionally a short audio clip) every 5–20 seconds as a background check job. Each check computes eight independent signals (Figure 3.8); a check is flagged if any signal fires, even when identity matches. Browser events (tab or focus loss, clipboard paste, a devtools heuristic, camera loss) are recorded separately. Enforcement is server-side: in the same database transaction as the check that crosses the exam's max_flags limit, the attempt is marked terminated_for_flags, so a modified client cannot keep submitting answers.

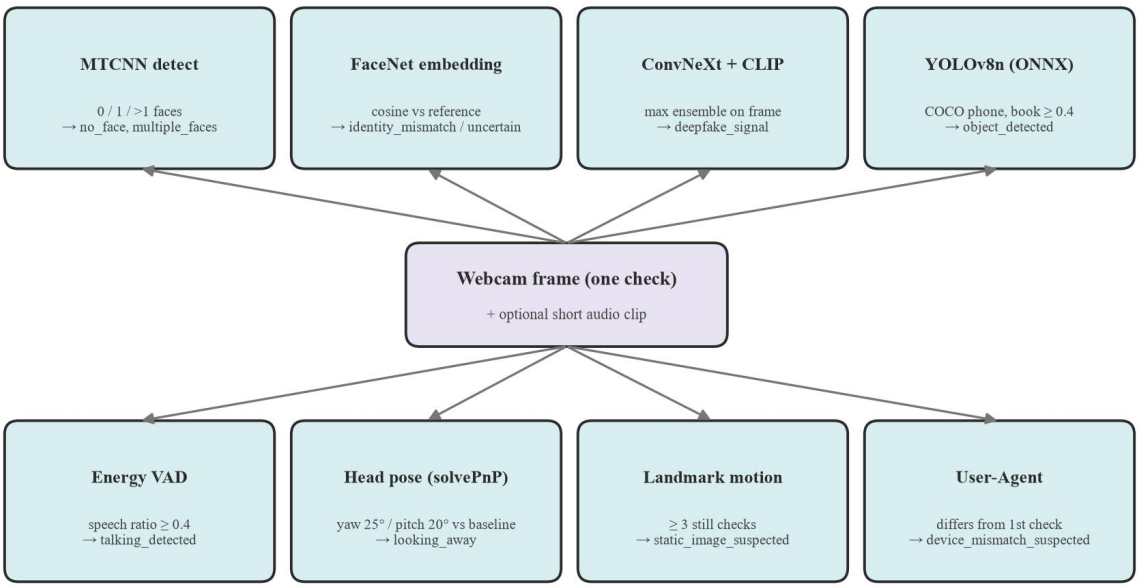


Figure 3.8: Signals computed for every monitoring check

Head pose is estimated from MTCNN's five landmarks with OpenCV's EPnP solver and compared with the session's own first good reading rather than a universal angle, because no labelled head-pose data was available to validate absolute angles. The liveness signal is not blink detection (five landmarks have no eyelid points); it flags a face whose landmarks stay almost perfectly still across three consecutive checks, as a printed photo would. Talking is detected by short-term-energy voice activity with an adaptive noise floor; it signals speech, not whose voice it is. These monitoring thresholds are stated placeholders, documented as such in the code.

3.3.8 Performance Metrics

The accuracy of a classifier is computed using Equation (3.6):

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \tag{3.6}$$

where TP, TN, FP and FN denote true positives, true negatives, false positives and false negatives respectively, with "fake" as the positive class. We also report fake recall (sensitivity), real specificity, precision and F1-score, Equations (3.7)–(3.9), ROC-AUC with 95% bootstrap confidence intervals (1,000 resamples), and the expected calibration error over ten equal-width bins, Equation (3.10). The metric code was verified against scikit-learn to within 2×10^{-16}.

$$Recall = \frac{TP}{TP+FN}, \;\; Specificity = \frac{TN}{TN+FP} \tag{3.7}$$

$$Precision = \frac{TP}{TP+FP} \tag{3.8}$$

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (3.9)$$

$$ECE = \sum_{b=1}^{10} \frac{n_b}{n} |acc(b) - conf(b)| \quad (3.10)$$

3.4 Tools and Technologies Used

- **Programming languages:** Python 3.11 (backend, models, evaluation), JavaScript/JSX (frontend).
- **Backend:** FastAPI with uvicorn, Pydantic, async SQLAlchemy over SQLite (aiosqlite), python-jose (HS256 JWT), passlib/bcrypt, ReportLab (PDF), c2pa-python, PyWavelets, PyAV, Pillow + pillow-heif.
- **Machine learning:** PyTorch 2.8 (CPU) and torchvision, OpenCLIP, Hugging Face Transformers (wav2vec 2.0), facenet-pytorch (MTCNN, InceptionResNet-v1), ONNX Runtime (AASIST, YOLOv8n), OpenCV, NumPy, scikit-learn.
- **Frontend:** React 18, React Router 7, Vite 5, Tailwind CSS 3, Axios, GSAP and Framer Motion (animation), MediaPipe Tasks Vision (live face-mesh overlay), Lucide icons.
- **Training:** Kaggle notebooks and Google Colab (free cloud GPUs) for fine-tuning; local CPU for evaluation.
- **DevOps:** Docker and Docker Compose (development and production), nginx 1.27, GitHub and GitHub Actions CI, pytest.
- **Hardware:** development laptops (Windows 11, CPU inference); deployment target is a single CPU server, about 265 MB resident memory after all image and video models are loaded.

Chapter 4

System Design and Implementation

This chapter turns the methodology into a working system: the requirements, the UML and data-flow views, the database and API design, the user interface, security, testing and deployment.

4.1 Requirements

4.1.1 Functional Requirements

- FR1: Users can sign up as a student or an organisation, log in and log out; sessions use 24-hour JWTs.
- FR2: Users can upload an image, video or audio file and receive a verdict, confidence, calibrated confidence and per-model sub-scores.
- FR3: Long video scans run in the background with real progress, queue position and cancel.
- FR4: Users can open an explanation and a provenance panel for any of their results and download a PDF report.
- FR5: Users can view their history and dashboard statistics, give "was this correct?" feedback with opt-in reuse consent, and delete their account with all files and reports.
- FR6: Organisations can create exams and questions, share a join code, manage the roster and review attempts, flag timelines and session reports.
- FR7: Students can join an organisation, verify their identity against a government ID and take a proctored exam that is scored on completion or terminated when the flag limit is reached.

4.1.2 Non-Functional Requirements

- **Privacy:** every upload, result, job, report, feedback item and session is visible only to its owner; others receive 404, never 403, so an identifier does not reveal

that a resource exists. GPS coordinates are never returned. Government-ID photos are deleted immediately after verification; only the face embedding is kept.

- **Honesty:** no fabricated numbers in the UI; borderline scores are UNCERTAIN; missing metadata is never presented as evidence of authenticity; every model claim carries its measured limits.
- **Responsiveness:** image and audio scans complete within the request (about 0.5–1.2 s and 3.5 s on CPU in the demonstrations in Chapter 5); video scans never block other users.
- **Security:** bcrypt password hashing, signed tokens, magic-byte upload validation, rate limits, CSP and refusal to start in production with a default secret.
- **Maintainability:** swappable model back-ends selected by environment variables, documented rollback paths, and an automated test suite run in CI.

4.1.3 Hardware and Software Requirements

Table 4.1 lists what is needed to run the system. The values come from the measured deployment (memory, file sizes) and from the configuration files in the repository; the team's own development and evaluation ran on ordinary Windows 11 laptops with CPU-only inference.

Table 4.1: Hardware and software requirements

Item	Requirement	Basis / notes
Processor	64-bit x86 CPU; no GPU needed for inference	All scans and evaluation in this report ran on CPU
Memory (RAM)	4 GB or more recommended	About 265 MB resident with image and video models loaded; PyTorch may commit up to about 2.3 GB of virtual memory
Storage	5 GB or more free	About 0.85 GB of project checkpoints, about 1.3 GB of downloaded CLIP and wav2vec 2.0 backbones, plus uploads and reports
GPU (training only)	Cloud GPU (Kaggle / Google Colab)	Used only for the fine-tuning rounds
Client device	Modern browser (Chrome, Edge or Firefox)	Webcam and microphone needed for ProctorLens exams
Network	Internet on first start	CLIP and wav2vec 2.0 backbones download once, then are cached
Operating system	Windows 10/11 (development); Linux via Docker (production)	docker-compose.yml and docker-compose.prod.yml
Backend runtime	Python 3.11	backend/Dockerfile.prod (python:3.11-slim)

Item	Requirement	Basis / notes
Frontend build	Node.js 20, npm	frontend/Dockerfile (node:20-alpine)
Web server	nginx 1.27	Serves the built SPA and proxies /api
Database	SQLite (any SQLAlchemy URL accepted)	DATABASE_URL in core/config.py
Containers	Docker Engine with Docker Compose	Optional for development, used for production

4.2 Use-Case Diagram

Figure 4.1 shows the actors and use cases. A user (or student) performs media verification and takes proctored exams; an organisation administers exams and reviews attempts; the periodic monitoring check is triggered by the exam page on the student's behalf.

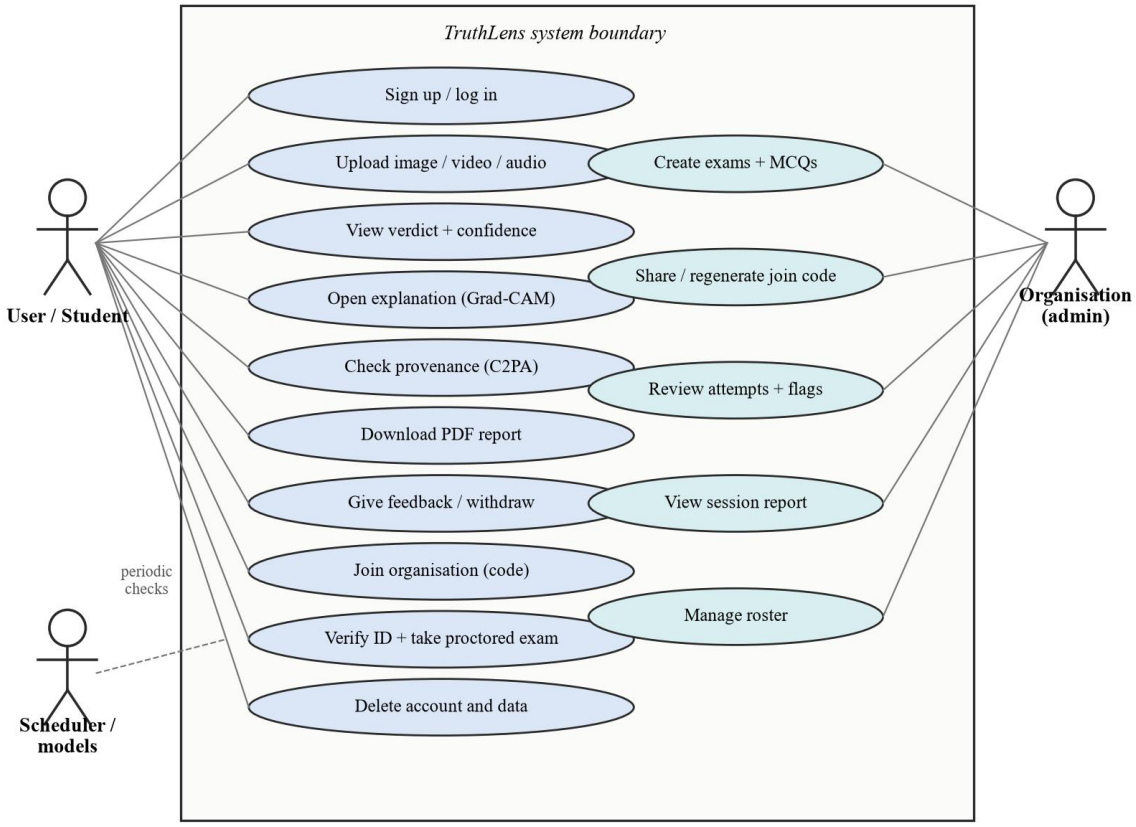


Figure 4.1: Use-case diagram

4.3 Data Flow Diagrams

Figure 4.2 is the context-level (level 0) DFD and Figure 4.3 the level 1 DFD, which decomposes the platform into seven processes and four data stores.

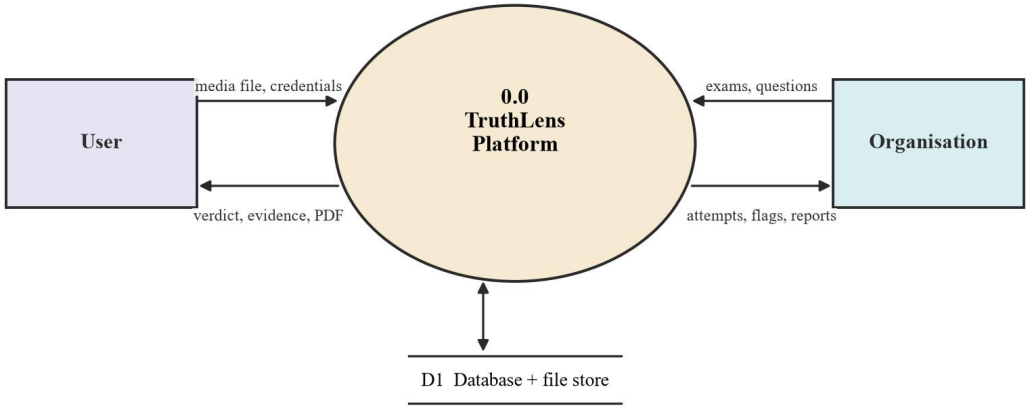


Figure 4.2: Data flow diagram, level 0

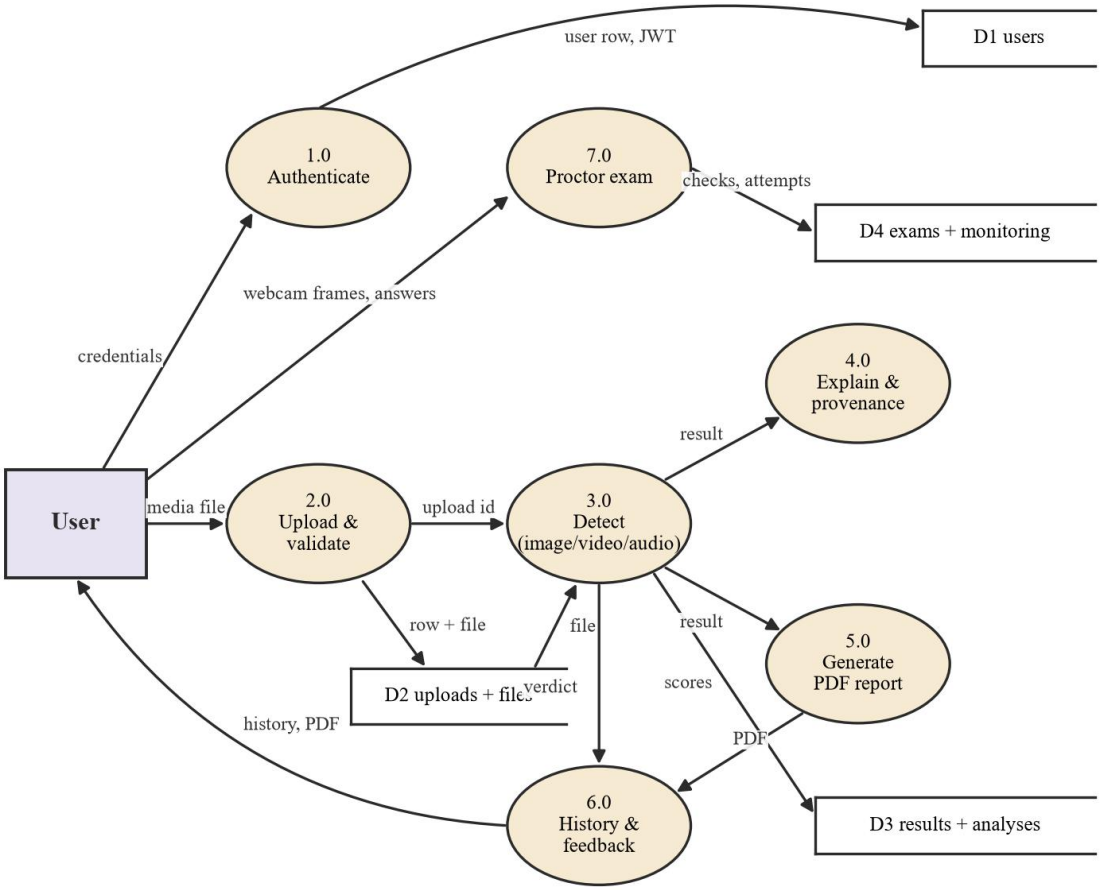


Figure 4.3: Data flow diagram, level 1

4.4 Sequence Diagrams

Figure 4.4 shows a synchronous scan of an image or audio file followed by the optional evidence and report requests. POST /detect is idempotent: an upload has at most one result. (A repeated call once stored a second row and broke every read endpoint for that

upload; a regression test now guards against it.) Figure 4.5 shows a video scan as a background job.

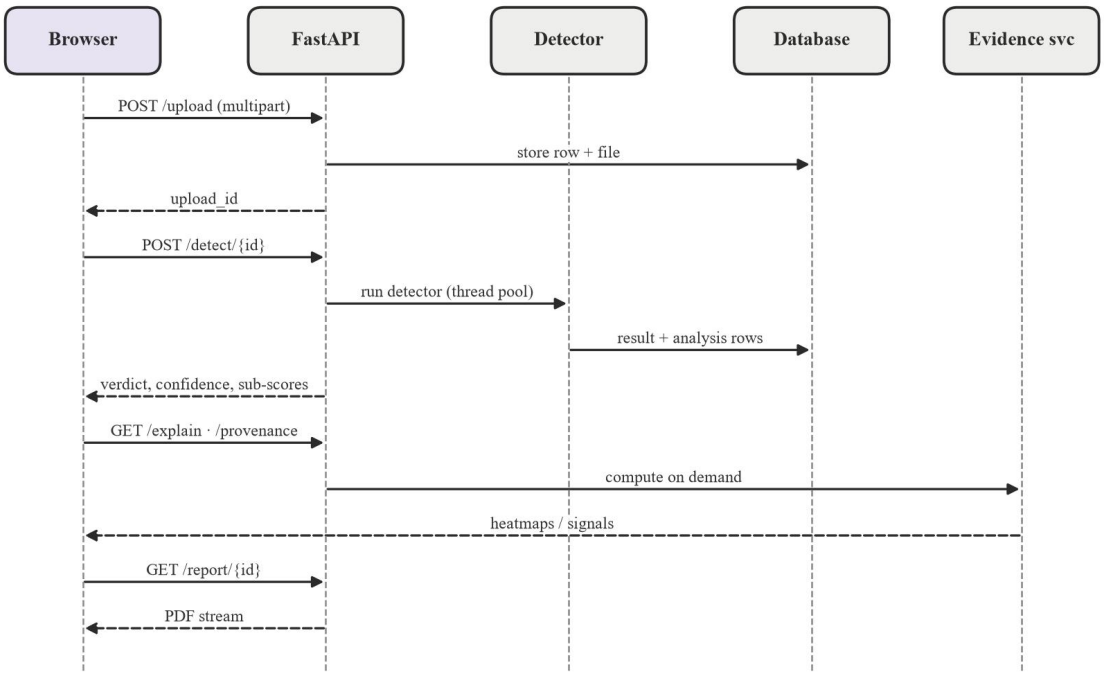


Figure 4.4: Sequence diagram: image or audio scan and evidence requests

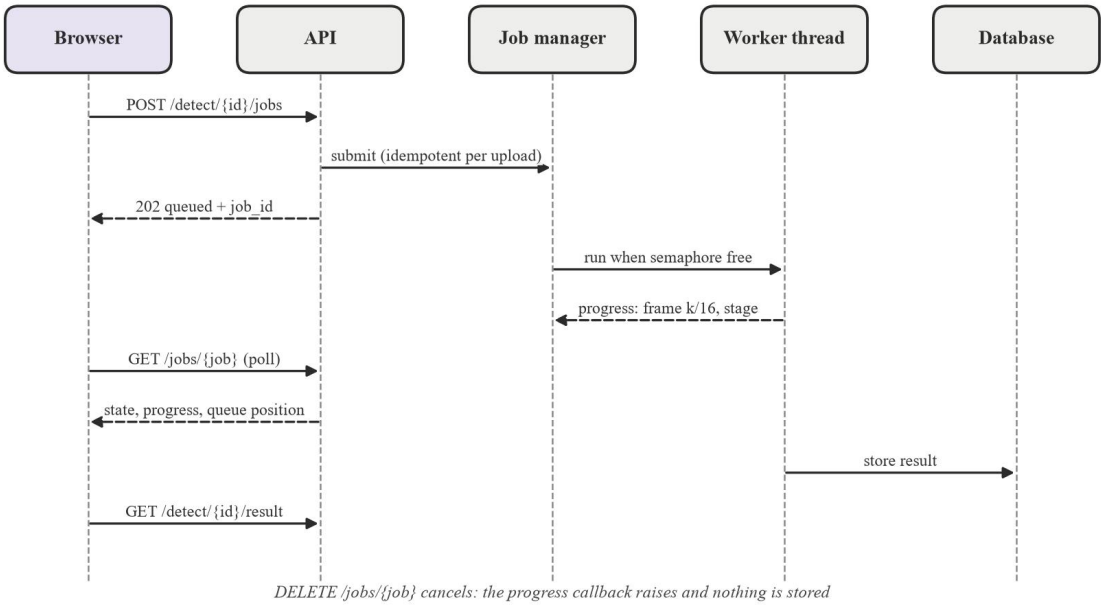


Figure 4.5: Sequence diagram: background video scan

Progress is real, not a timer: frame decoding reports 0–70%, face cropping 70–90% and scoring the rest. The callbacks are observation-only, and predictions were verified bit-identical with and without them. Cancelling raises an exception inside the callback, so the

scan stops and nothing is stored. At most DETECTION_JOB_CONCURRENCY jobs run at once (default one, since inference is CPU-bound).

4.5 Database Design

Figure 4.6 shows the entity-relationship diagram. A detection result belongs to exactly one upload and has one modality-specific analysis row. Feedback is unique per (upload, user). The audit log deliberately has no foreign key to users, so an "account deleted" entry survives the account it describes. An exam attempt is linked one to one with the monitoring session that proctors it. New nullable columns are added by light, idempotent migrations at start-up.

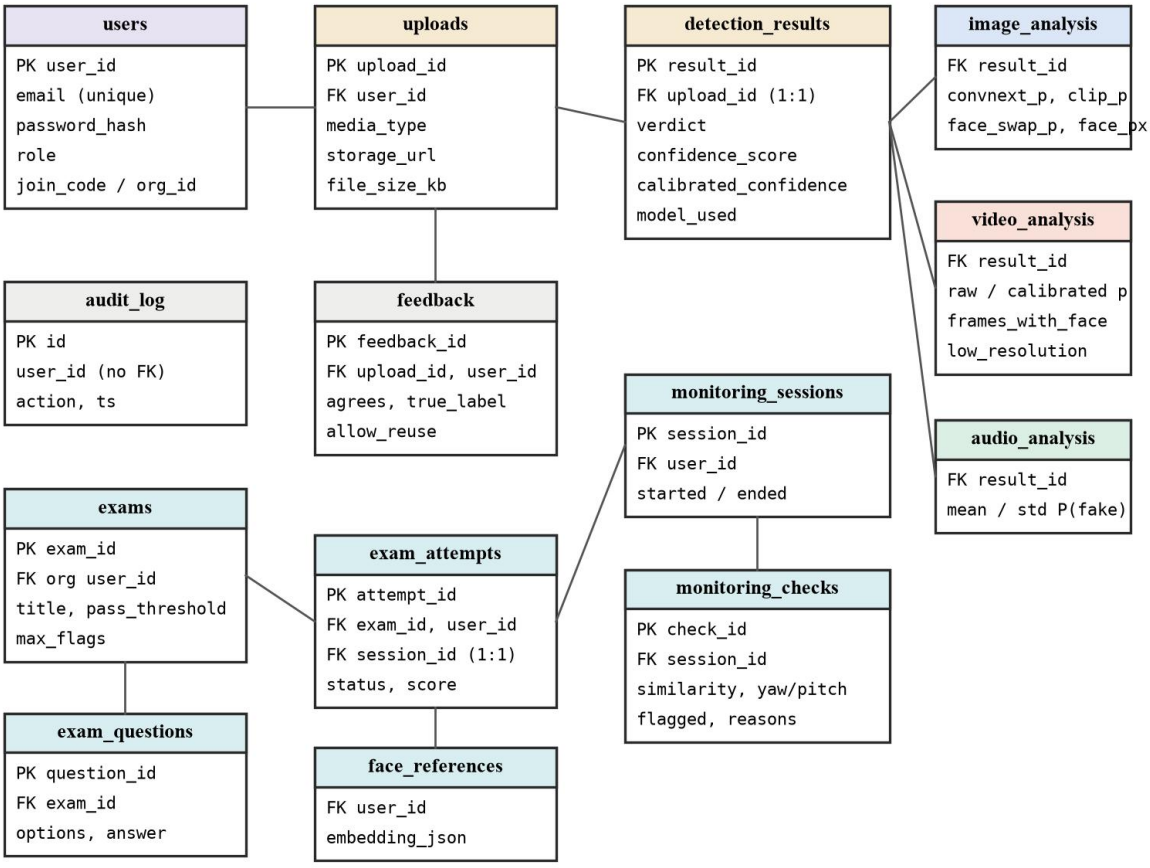


Figure 4.6: Entity-relationship diagram

4.6 API Design

All routes are mounted under /api/v1 and documented automatically by FastAPI's OpenAPI page. Table 4.2 lists the main endpoints.

Table 4.2: Main REST endpoints

Method and path	Purpose
POST /auth/signup, /auth/login; GET, DELETE /auth/me	Account creation, login (JWT), profile, full account deletion
POST /upload/; GET /upload/{id}	Validated upload (size, MIME, magic bytes); upload metadata
POST /detect/{id}; GET /detect/{id}/result	Synchronous detection (idempotent); stored result
POST /detect/{id}/jobs; GET, DELETE /jobs/{job}	Background video scan with progress; poll or cancel
GET /detect/{id}/explain; /provenance	Grad-CAM, per-frame or audio saliency; C2PA, metadata, watermark, ELA
GET, PUT, DELETE /detect/{id}/feedback	"Was this correct?" feedback with consent; withdraw
GET /report/{id}; /history; /stats	PDF report; history list; dashboard counts
POST /identity/verify-government-id; /match	Government-ID enrolment; one-shot identity match
POST /identity/sessions/{id}/checks[/jobs]; /events	Monitoring checks (sync or job); browser events
POST /exams; /exams/{id}/questions; GET /exams/mine	Exam and question management; exams for a student
POST /exams/{id}/attempts/verify-identity; /attempts	Per-attempt identity check; start attempt
POST /exams/attempts/{id}/answers; /finish	Record answers; score and close the attempt
GET /admin/join-code, /roster, /sessions, /exams/{id}/attempts	Organisation administration and review
GET /health; /health/ready	Liveness; readiness (database + every checkpoint present, else 503)

4.7 Frontend Implementation

The client is a React single-page application. App.jsx defines the live routes (home, verify, history, reports, report detail, dashboard, ProctorLens, exam attempt and result, admin, about, login and sign-up), protected by ProtectedRoute and GuestRoute wrappers; every page is lazily loaded, which reduced the main bundle from 566 KB to 447 KB. A single Axios client attaches the bearer token to every request, converts errors into friendly messages that include the server's request ID, and redirects to a clear "session expired" message on a 401. The visual language is a "forensic evidence bench" (dark ground, bone text, brass accents, serif headings). Key components are UploadFlow (the scan flow), ExplanationPanel (lazy, fetched only when opened), ProvenancePanel, FeedbackPanel,

LowResolutionNote and the ProctorLens components (GovernmentIdCapture, WebcamCapture and a live face-mesh overlay).

4.8 PDF Report Generation

The report generator (ReportLab) produces a forensic report for every result: a verdict banner written for a non-expert, a file and confidence table, technical details (model, timing, per-model probabilities, frames with a detected face, resolution), a scope note for video, an explainability section with the same images as the interface, and a provenance section. Explainability and provenance are best-effort: a failure in either only skips that section. A separate session report summarises a proctored attempt.

4.9 Security and Privacy

- Passwords are hashed with bcrypt; tokens are HS256 JWTs with a 24-hour expiry. In production mode the server refuses to start if the signing key is the public development default or shorter than 32 characters.
- Uploads are validated by declared MIME type and by their leading bytes, so a renamed executable is rejected. HEIC images are supported end to end.
- Ownership is checked on every route and answered with 404 for other users' data.
- The production nginx configuration adds a Content-Security-Policy derived from auditing the built bundle, security headers, and per-IP rate limits of 10 requests per minute on authentication and 20 per minute on upload and detection.
- Security-relevant actions (sign-up, login, failed login, feedback withdrawal, account deletion, feedback export) are written to an audit log; every response carries an X-Request-ID that is echoed in error messages.
- Feedback files are kept for evaluation only with explicit, per-item, withdrawable consent (off by default). Users can delete their account, which removes every upload, report, result and feedback row.

4.10 Data Protection and Ethical Use

ProctorLens processes biometric data (face images and embeddings) and photographs of government IDs, and TruthLens stores the files users upload. In India such data falls under the Digital Personal Data Protection Act, 2023, which expects consent for a stated

purpose, collection limited to what that purpose needs, reasonable security safeguards and erasure on request. The implementation follows these principles as follows:

- **Consent:** identity enrolment and government-ID verification refuse to proceed without explicit consent, and the time of consent is stored; reuse of feedback files for evaluation is a separate, opt-in, withdrawable consent.
- **Data minimisation:** the government-ID photo and the verification selfie are deleted right after the embedding is extracted, on success and on failure; GPS coordinates found in uploads are never returned; a retention script deletes raw monitoring frames older than a chosen number of days while keeping the derived flags needed for reports.
- **Security safeguards:** owner-only access on every route, hashed passwords, signed tokens, rate limits and an audit log of security-relevant actions.
- **Erasure:** a user can delete the stored face reference, and deleting the account removes every upload, report, result and feedback record.
- **Human oversight:** proctoring flags and detection verdicts are presented as evidence for a person to review; the only automatic action is ending an exam attempt at the flag limit that the organisation itself sets.

These are engineering measures, not a legal compliance audit. An institution deploying the system would still need a privacy notice for candidates, a fixed retention period (the retention script must be scheduled; it does not run by itself), a grievance contact and a review by its data-protection officer. Uploaded media in TruthLens is kept until the user deletes it or the account.

4.11 Testing

The backend has a pytest suite of 46 test modules. In the final run on the development machine 530 tests passed and 1 was skipped (a check whose test images are not present); tests that need a model checkpoint skip themselves when it is missing. The CI subset (pytest -m "not models") needs no model files and runs on GitHub Actions together with a clean frontend build. The suite covers authentication, upload validation, 404-not-403 ownership on every route, error mapping, verdict banding including the protected thresholds, calibration, jobs (non-blocking, cancel, queue position), explainability and provenance logic (tamper, strip, hostile metadata), the watermark decoder, PDF content,

feedback consent, deployment helpers, identity, monitoring, exams and the evaluation metrics against scikit-learn. Checkpoint-dependent tests reproduce production scores exactly. A mutation script deliberately breaks 17 guarded behaviours (for example the thresholds, ownership checks, idempotent detect and consent gating) and confirms that the named test fails each time. Table 4.3 lists representative test cases; each one is an automated test in backend/tests, and all of them passed in the final run of the suite.

Table 4.3: Representative test cases

ID	Module	Input / action	Expected result	Result
TC01	Authentication	Sign up with a password shorter than 8 characters	Rejected with a validation error	Pass
TC02	Authentication	Log in with a wrong password, and with an unknown email	Same 401 error in both cases	Pass
TC03	Authentication	Call /auth/me with a tampered, wrongly signed or expired token	Rejected with 401	Pass
TC04	Upload	Plain text sent with Content-Type image/png	Rejected: leading bytes do not match	Pass
TC05	Upload	File larger than MAX_FILE_SIZE_MB	Rejected; nothing stored	Pass
TC06	Access control	Second user requests the first user's upload, result, report, explanation or provenance	404 Not Found (not 403)	Pass
TC07	Detection	POST /detect twice for the same upload	One stored result; all read endpoints keep working	Pass
TC08	Verdict bands	Scores at and around 0.3, 0.5 and 0.7	REAL / UNCERTAIN / FAKE exactly as specified	Pass
TC09	Background jobs	Cancel a running video job	Job stops early; no result stored	Pass
TC10	Background jobs	Other API requests while a video scan runs	Served without waiting for the scan	Pass
TC11	Provenance	Change one byte of a C2PA-signed image	Reported as changed since signing	Pass
TC12	Provenance	JPEG with GPS EXIF	GPS presence reported; coordinates never returned	Pass
TC13	Feedback	Export feedback from users with and without reuse consent	Files and comments exported only for consenting users	Pass
TC14	Account	Delete account	Uploads, files, results and feedback removed; other users untouched	Pass
TC15	Identity	Verify a government ID (accepted and rejected by the quality gate)	ID upload file and row deleted in both cases	Pass
TC16	Exams	Student opens the question list	Answer key never included	Pass
TC17	Exams	Flagged checks reach the exam's max_flags (synchronous and job paths)	Attempt terminated on the server	Pass

ID	Module	Input / action	Expected result	Result
TC18	Monitoring	Identity matches but the frame scores as a deepfake	Check is still flagged	Pass

4.12 Deployment

Figure 4.7 shows the production deployment. docker-compose.prod.yml runs a frontend container (nginx serving the built SPA and proxying /api) and a backend container (non-root, a single uvicorn process without reload, DEBUG=false). State lives on a named volume; model checkpoints are mounted read-only and verified against SHA-256 checksums by a fetch script that discards any mismatched download. A readiness probe returns 503 until the database and every checkpoint are available.

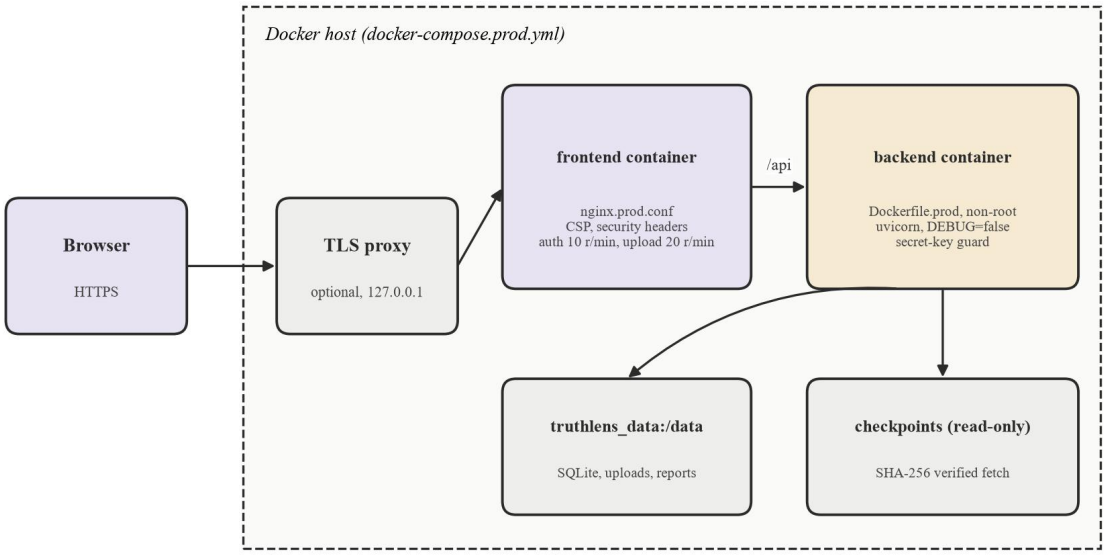


Figure 4.7: Production deployment

4.13 Project Timeline

Figure 4.8 shows the implementation phases recorded in the repository history (47 commits by the team between 3 September and 19 October). Earlier work, such as the problem definition, the literature survey and the first CIFAKE image model, was done before the repository was created; its dates were not recorded, so it is not drawn rather than drawn with guessed dates.

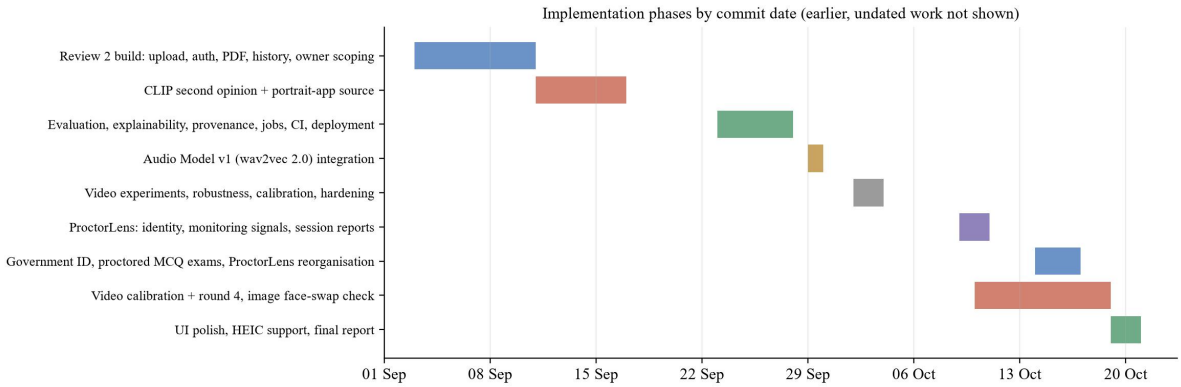


Figure 4.8: Implementation timeline

Chapter 5
Results and Discussion

This chapter presents the measured performance of the deployed system and screenshots of it at work. All measurements use the production detector classes with unchanged weights and thresholds, and every data source is tagged seen or unseen so that fit and generalisation are never mixed.

5.1 Experimental Setup

Evaluation ran on CPU on the team's Windows 11 development machines using the harness in backend/eval (build_manifest → run_predictions → make_report). Training and fine-tuning ran on Kaggle and Colab GPUs. Image evaluation used 6,532 files from twelve sources (up to 500 per class per dataset); video used 300 unseen RTFS clips plus ten team videos for the epoch-11 model, and 356 held-out videos for the round-4 comparison. Robustness used 300 label-balanced images (180 seen, 120 unseen) under 21 settings, following the corruption methodology of [36], and 30 RTFS clips under 11 settings for video. Identity used 1,000 LFW pairs; fairness used 600 UTKFace [37] portraits, 60 per gender-race group.

5.2 Results

5.2.1 Image Detection: Seen Versus Unseen Sources

Figure 5.1 and Table 5.1 contain the central result of this work. On seen sources every configuration is near perfect. On unseen generators the deployed ensemble catches only 27.3% of fakes, and AUC drops to 0.665.

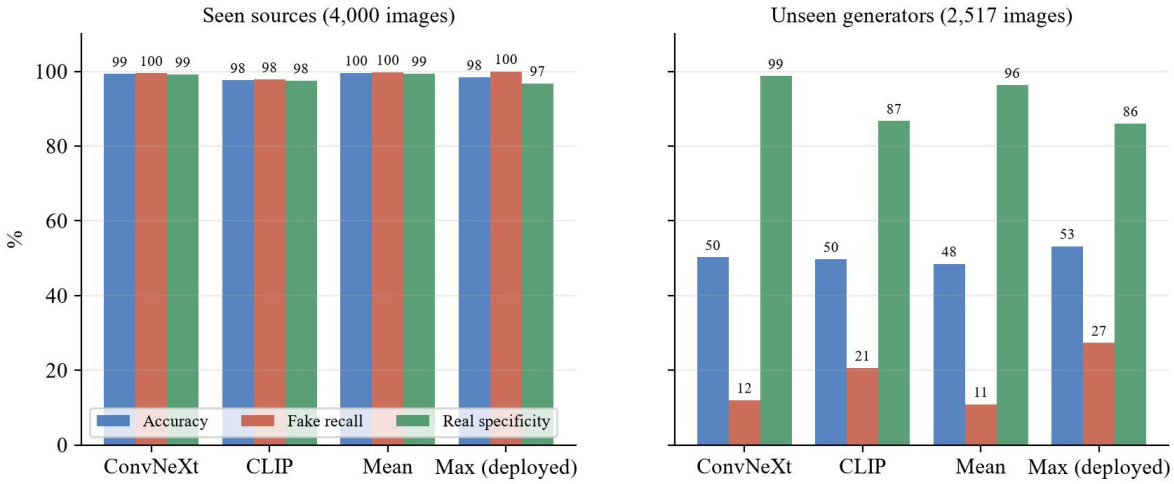


Figure 5.1: Image detection by training status of the source (threshold 0.5)

Table 5.1: Image detection by training status of the source

System	Data	Accuracy (%)	Fake recall (%)	Real specificity (%)	AUC
ConvNeXt	Seen	99.3	99.5	99.2	1.000
CLIP	Seen	97.6	97.8	97.5	0.997
Mean of both	Seen	99.6	99.8	99.4	1.000
Max (deployed)	Seen	98.4	100.0	96.8	1.000
ConvNeXt	Unseen	50.2	12.0	98.7	0.665
CLIP	Unseen	49.7	20.6	86.7	0.640
Mean of both	Unseen	48.5	10.9	96.3	0.670
Max (deployed)	Unseen	53.2	27.3	86.0	0.665

Per source (Figure 5.3), the deployed system catches 30.7% of GenImage Stable Diffusion images, 20.0% of Midjourney and 26.0% of BigGAN images, and reaches 54.3% accuracy on OpenFake. All eight photorealistic ChatGPT "everyday" images were labelled REAL with more than 99.99% confidence. The maximum rule trades specificity for recall: compared with the mean it raises unseen-fake recall from 10.9% to 27.3% but lowers real-photo specificity from 96.3% to 86.0%; on the portrait-app genre it keeps CLIP's 93.3%. Figure 5.4 shows the ROC and precision-recall curves over all 6,532 images.

Figure 5.2 shows the confusion matrices of the deployed detectors. For images, the binary matrix at threshold 0.5 is pooled over all 6,532 evaluation files; almost all of the 1,026 missed fakes come from the unseen generators. The three-way matrix shows how the UNCERTAIN band absorbs borderline scores (271 files). The video matrices are for the epoch-11 model on 310 unseen clips: at threshold 0.525 it separates real from fake

reasonably well, but under the original raw-score band 304 of 310 clips were UNCERTAIN, which is the problem the Platt calibration in Section 3.3.2 solves.

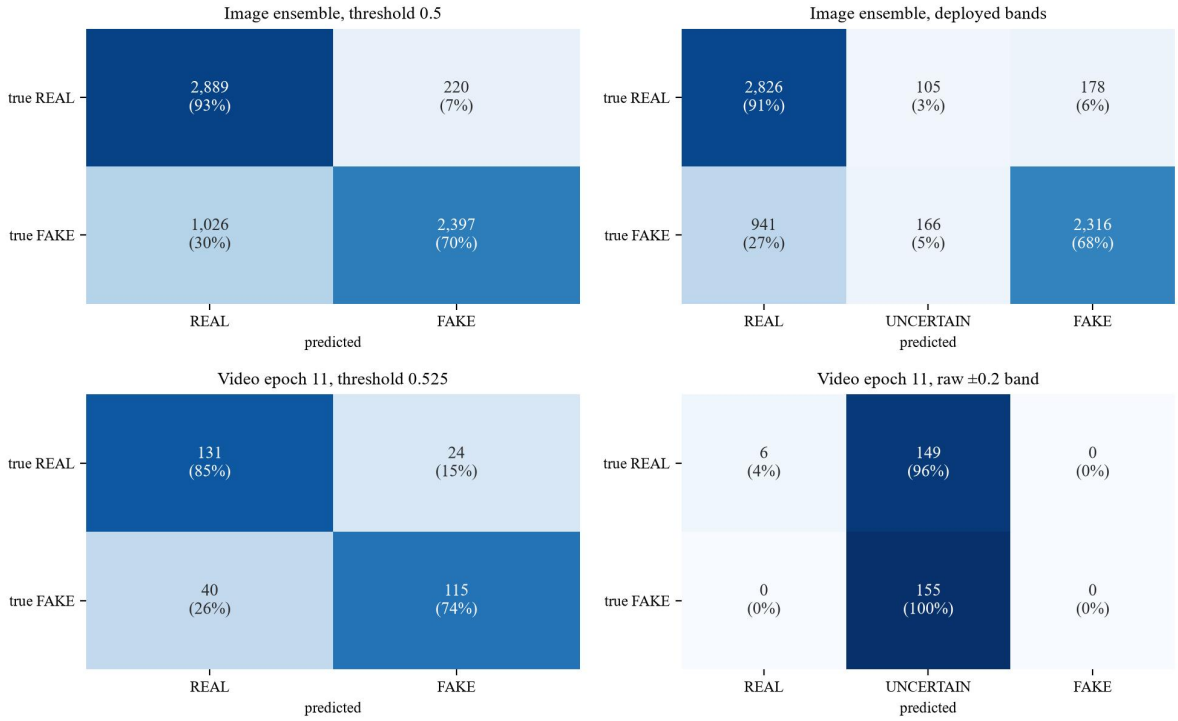


Figure 5.2: Confusion matrices of the image ensemble (6,532 files) and the epoch-11 video model (310 clips)

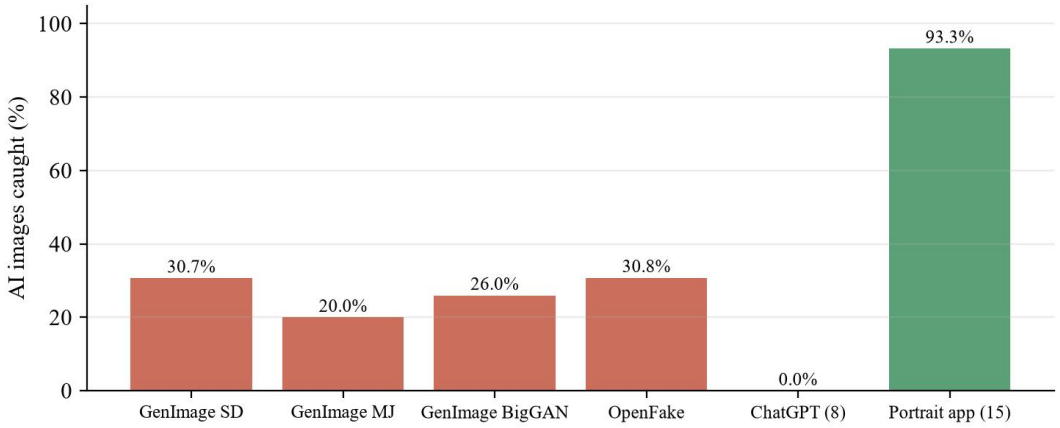


Figure 5.3: Share of AI images caught per unseen source (deployed max ensemble)

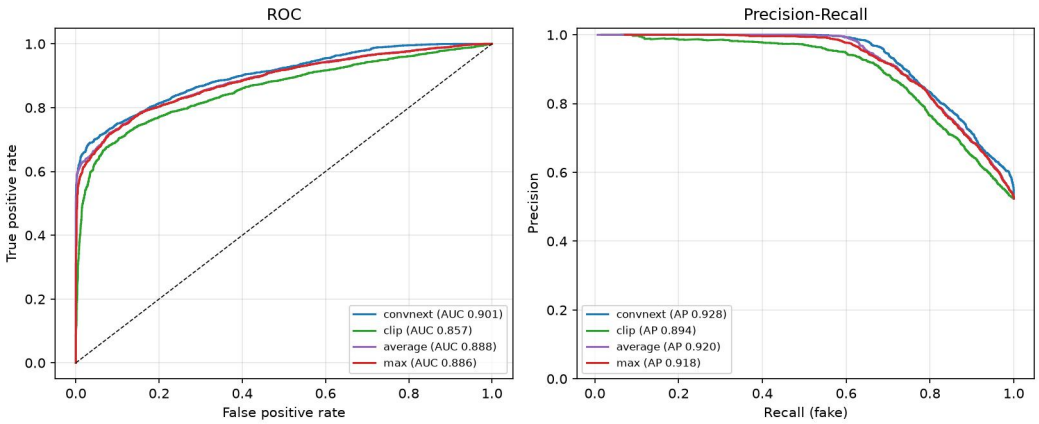


Figure 5.4: ROC and precision-recall curves of the image sub-models and ensembles (6,532 images)

5.2.2 Score Distributions, Threshold Sensitivity and Reliability

Figure 5.5 shows how the deployed ensemble's fake probability is distributed for real and fake images. Both classes pile up at the extremes: most real images score close to 0 and most fakes close to 1, so the UNCERTAIN band between 0.3 and 0.7 holds few files. The problem is the several hundred fakes, almost all from unseen generators, that also score close to 0. Those images are not borderline; the model is confidently wrong about them.

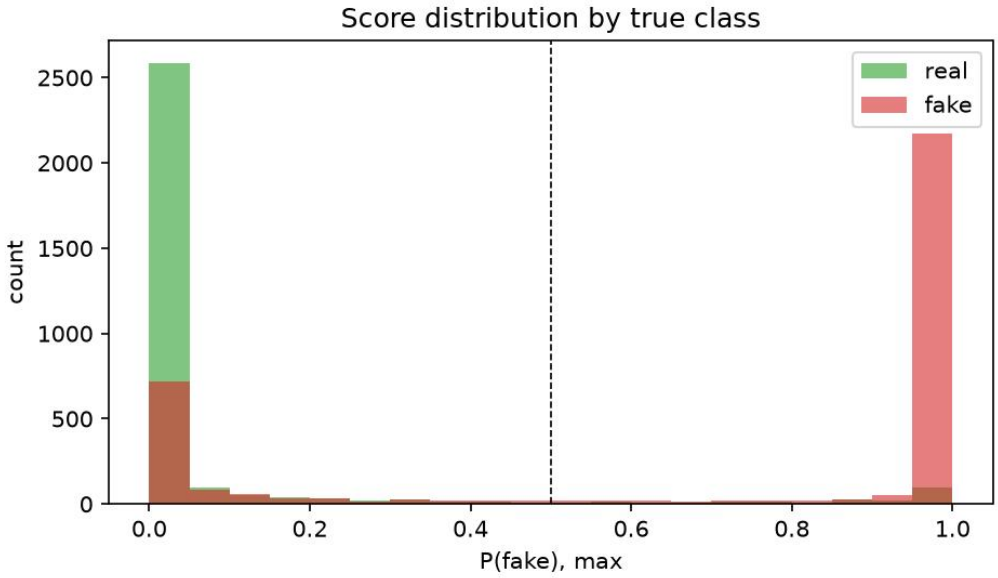


Figure 5.5: Distribution of the deployed image ensemble's fake probability by true class (6,532 images)

This is also why changing the decision threshold does not rescue the unseen generators (Figure 5.6). Balanced accuracy of the deployed maximum rule stays between about 0.80 and 0.82 for every threshold from 0.05 to 0.95, and at the chosen 0.5 it is already near its best. ConvNeXt alone does slightly better at very low thresholds because a low threshold

also flags fakes that it scores only a little above zero. The 0.5 threshold was not tuned on the evaluation data, so these numbers are not optimistic in that respect.

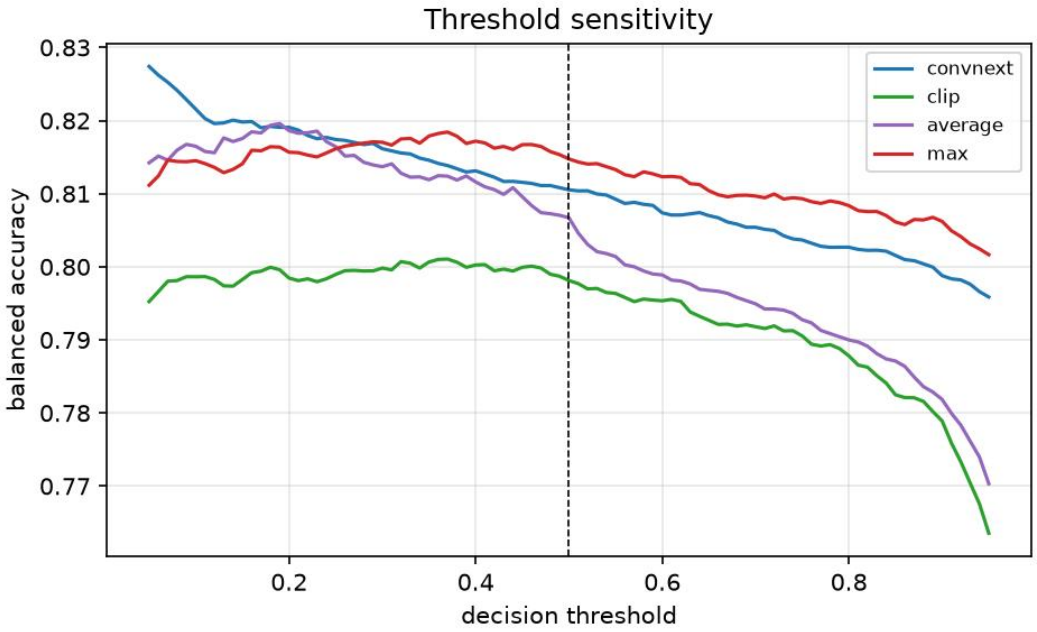


Figure 5.6: Balanced accuracy of each image configuration against the decision threshold

The reliability diagram (Figure 5.7) compares predicted fake probability with the observed share of fakes in ten bins. Images scored near 0 are in fact fake about 23% of the time, and images scored 0.8–0.9 only about 61% of the time, so the raw ensemble is over-confident at both ends (ECE 0.159 in this run). This is the measurement that motivated showing a separately calibrated confidence next to the raw one (Section 3.3.4).

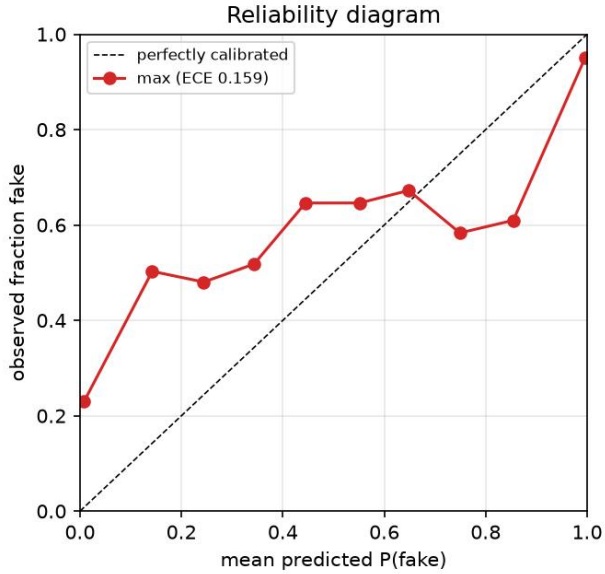


Figure 5.7: Reliability diagram of the raw image ensemble score

5.2.3 Face-Swap Check in the Image Pipeline

Figure 5.8 compares the face-swap check with the whole-image models. On frames from 356 held-out research videos the face check reaches AUC 0.95 against 0.61 for the image ensemble, and at a 5% false-alarm budget it catches 78% of fake frames against 12%. On inswapper swaps of 80 CelebA photos it reaches 0.859 against 0.659. On RWFS, 500 swaps from eight real online face-swap apps, the first version was near chance (0.617), which prompted photo model round 2: in a leave-one-group-out test its held-out-group AUC rose to 0.77–0.83 while keeping the research-frame AUC at 0.931 and the share of real faces wrongly flagged at 4.9%. These figures were measured on research datasets; no photos from the specific consumer apps users may encounter were available for testing.

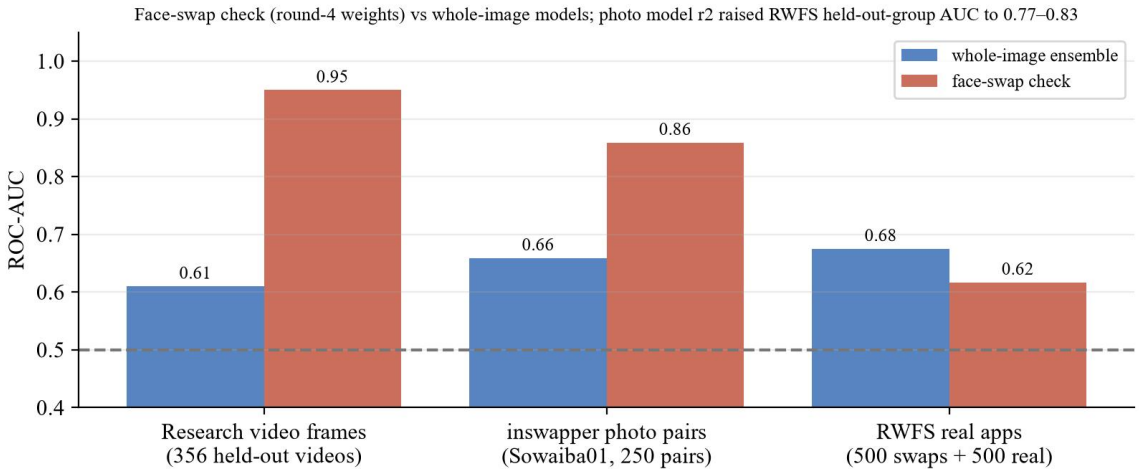


Figure 5.8: Face-swap check versus whole-image models (ROC-AUC)

5.2.4 Video Detection

With the original epoch-11 weights, the video model reached 79.4% accuracy and AUC 0.892 [0.852, 0.926] on 310 unseen clips (RTFS plus ten team videos), catching 77.3% of inswapper and 72.0% of uniface swaps while keeping 84.0% of real originals real. Inspecting the face crops explained the two consumer-app misses: in the Akool clip the Haar cascade locks onto a wall poster and a wrist, and the Magic Hour clip is stored rotated by 90° with no rotation flag, so no face is found in any frame. Three preprocessing fixes were tested on all 300 RTFS clips (Figure 5.9): a rotation fallback that only activates when no frame has a face (safe, but it never activated on RTFS), a two-cascade consensus (removes the poster detections but also good ones, AUC 0.890) and wider 35% padding (AUC 0.849, 5.3 points lower specificity). The existing crop geometry is close to a local optimum, so further gains needed retraining.

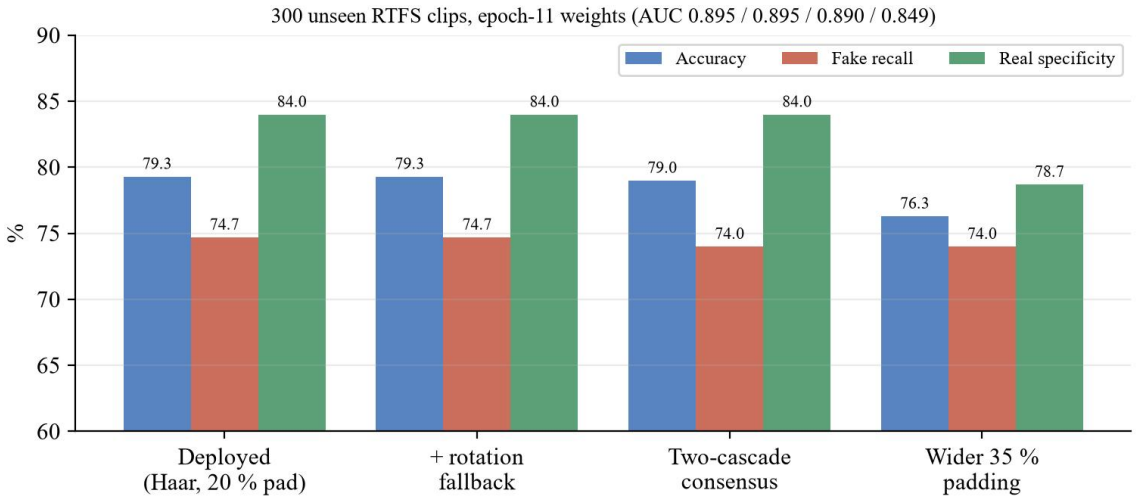


Figure 5.9: Video preprocessing variants on 300 unseen RTFS clips

Figure 5.10 shows the ROC and precision-recall curves of the epoch-11 model on the same 310 unseen clips (AUC 0.892, average precision 0.882). The curve rises steeply at low false-positive rates: about 70% of fakes can be caught while flagging about 10% of real videos, which is the operating region the calibrated band later made reachable.

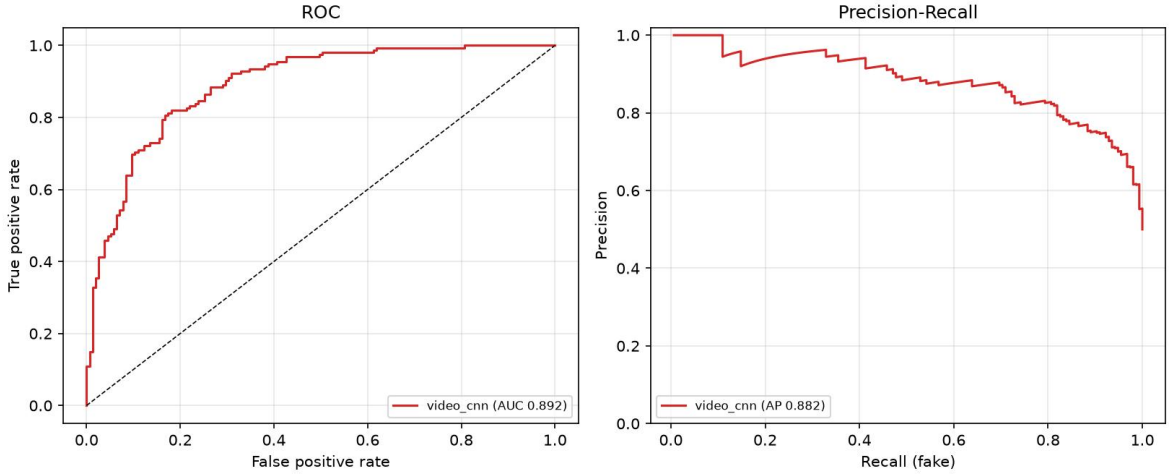


Figure 5.10: ROC and precision-recall curves of Video Model v1 (epoch 11) on 310 unseen clips

The raw scores of the epoch-11 model sat almost entirely between 0.2 and 0.7, so the original ±0.2 band around 0.525 made 294 of 300 RTFS videos UNCERTAIN. Platt rescaling of the mean logit, fitted on a grouped split, raised the share of decided RTFS test videos from 1.7% to 75% with 91.1% accuracy among them. Figure 5.11 shows why the resolution guard was added: AUC falls from 0.923 at 1080p to 0.729 at 240 px, and FakeAVCeleb's 224 px clips are at chance; without the guard the rescaling produced confident wrong verdicts on such clips.

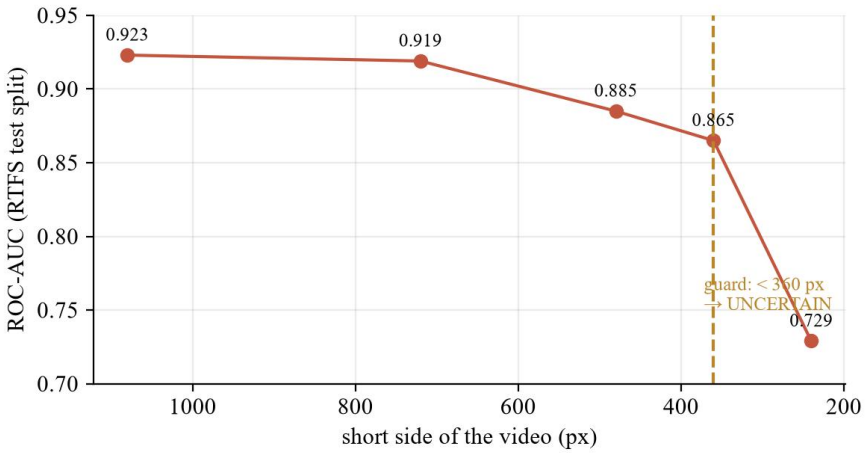


Figure 5.11: Video ROC-AUC against the short side of the video

Round 4 fine-tuning (Figure 5.12) raised the AUC on 356 local held-out videos from 0.765 to 0.961, reduced real videos wrongly flagged FAKE from 21 to 8 of 185 and increased fakes caught from 82 to 114 of 171; on held-out DFD and phone actors the AUC rose from 0.845 to 0.977. Fully AI-generated video (five Sora/Kling clips) is out of scope: none were caught, and the PDF report says so.

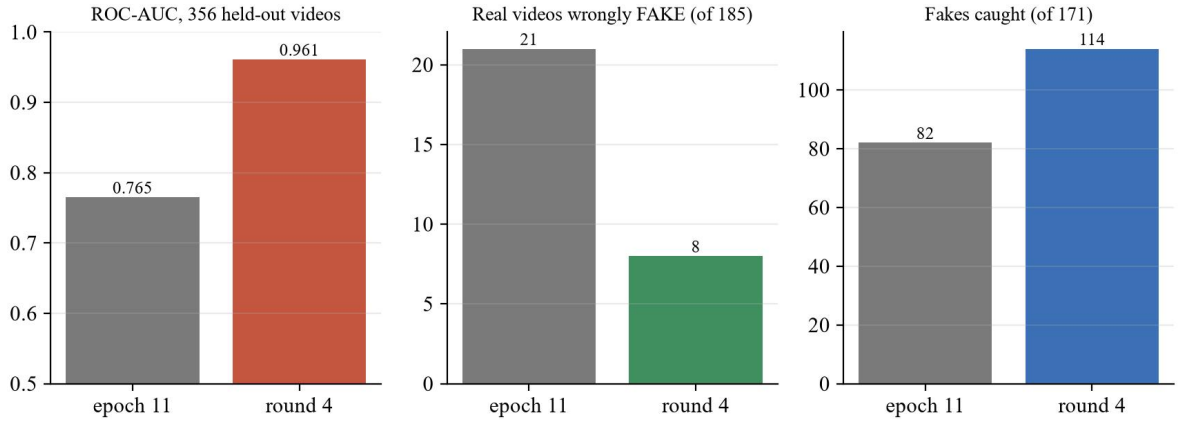


Figure 5.12: Video Model v1: epoch 11 versus round 4 on held-out videos

5.2.5 Audio Detection

The audio model was assessed on the team's six held-out acceptance sets during fine-tuning and passed five of the six, including 10 of 10 genuine clips from previously unseen LibriSpeech speakers (Figure 5.13). On a stress test with one clip from each of the 59 VCC 2020 voice-conversion systems, the previous model version scored 30/59 (50.8%, essentially chance); adding a system-split VCC source raised this to 98.3%. Because about 40 of those systems were used in training, the figure is partly a measure of fit. These results come from the training notebooks and have not yet been reproduced in the

evaluation harness, and audio robustness to compression and noise has not been measured, because no labelled audio dataset was available locally.

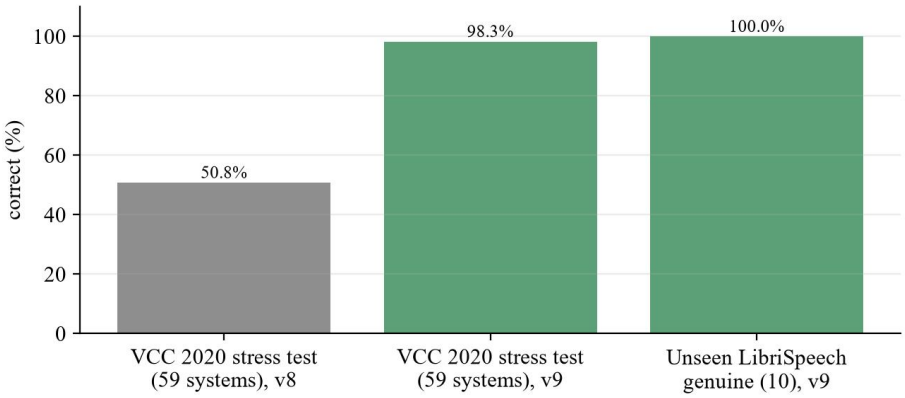


Figure 5.13: Audio model results on held-out acceptance sets

5.2.6 Robustness to Everyday Degradation

Common sharing pipelines leave the image ensemble almost unaffected on seen sources: screenshot, WhatsApp-style (longest side 1600 px, JPEG quality 65) and WebP processing all keep AUC at 0.99 or above. Figure 5.14 shows the exceptions. CLIP raises false alarms on real photographs under faint noise and compression: the ensemble's real-photo specificity falls from 95.6% clean to 87.8% at JPEG quality 90, 71.1% at noise σ = 5 and 62.2% at quality 10, while ConvNeXt alone stays between 96.7% and 98.9% in all 21 settings. Under heavy blur (σ = 3) the opposite happens: ConvNeXt catches only 50% of fakes and the ensemble recovers to 83.3% because CLIP still detects them. The maximum rule therefore inherits CLIP's false alarms and ConvNeXt's misses under different degradations.

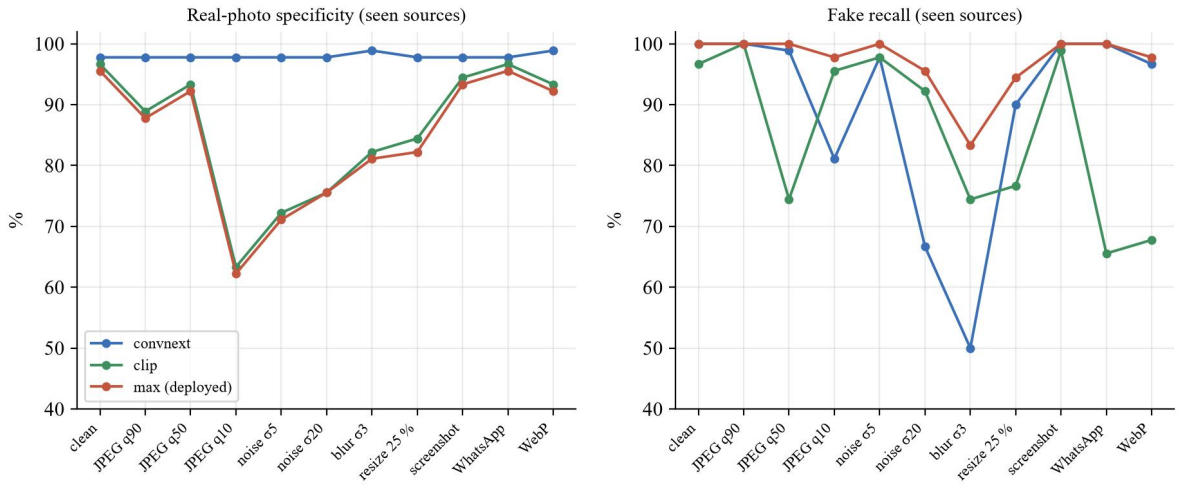


Figure 5.14: Image robustness on seen sources: specificity and recall under degradation

Figure 5.15 gives the complete picture for all 21 settings, separately for seen and unseen sources. Seen-source AUC stays at 0.91 or above in every setting, while unseen-source AUC stays between 0.50 and 0.62 whatever is done to the image: degradation does not create the generalisation gap, it only moves scores around inside it. The unseen fake-recall column rises under heavy JPEG compression (0.17 clean to 0.47 at quality 10) only because compression pushes every score towards FAKE, which the falling real-specificity columns confirm.

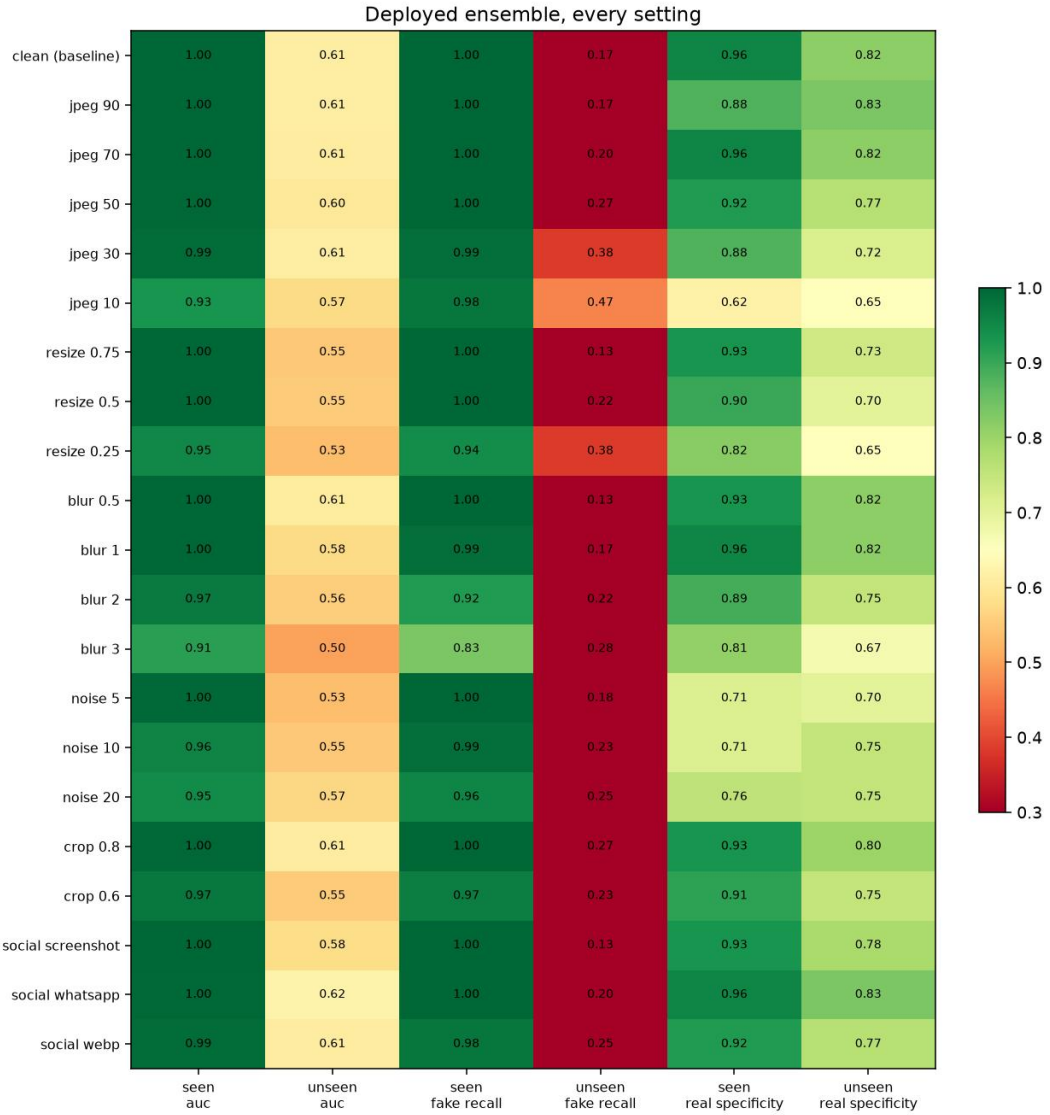


Figure 5.15: Deployed image ensemble under all 21 robustness settings (AUC, fake recall and real specificity, seen and unseen sources)

Video was far more fragile (Figure 5.16, epoch-11 weights, 30 clips). At JPEG quality 10 every degraded fake was labelled REAL; blur $\sigma = 3$ and 4× down-scaling pushed AUC below 0.5, meaning the score pointed the wrong way; WhatsApp-style processing alone

dropped AUC to 0.61. This finding directly motivated the degradation augmentation used in round 4; the robustness study has not yet been repeated for round 4.

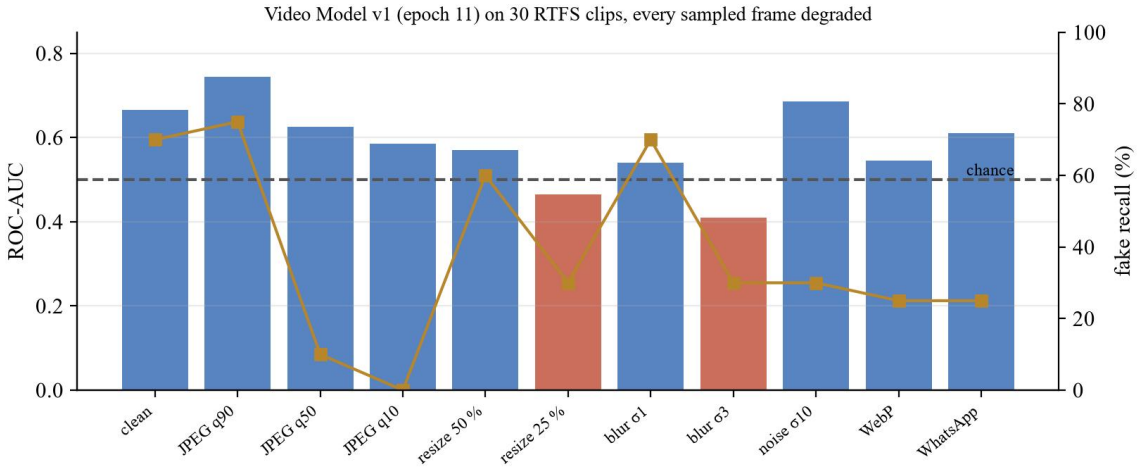


Figure 5.16: Video robustness with every sampled frame degraded (30 RTFS clips)

5.2.7 Explanations, Provenance and Calibration

Faithfulness. For 195 fakes that ConvNeXt scores FAKE, masking the 20% of the image that Grad-CAM ranks highest lowered the fake probability by 0.232 on average, against 0.044 for a random mask of the same area; the Grad-CAM mask had the larger effect on 85% of images, so the maps do point at regions the score depends on.

Provenance. Across the 6,532 evaluation images only 0.4% of AI images carry any AI declaration, because benchmark datasets are re-encoded and stripped. The signal is valuable exactly where the detectors fail, however: all eight ChatGPT images carry valid, unmodified OpenAI Content Credentials declaring AI generation, and in total 9 of the 1,026 AI images the ensemble misses declare themselves AI-generated. The declaration survived a byte-for-byte copy in 8 of 8 cases and every re-encode we tried in 0 of 8. ELA statistics separate real from fake with AUC 0.53–0.75 on most sources but 0.995 on DeepDetect, which reveals a JPEG-history shortcut in that dataset.

Calibration. Both detectors are mis-calibrated in raw form (Figure 5.17). Temperature scaling lowered the image ECE from 0.158 to 0.056 and the video ECE from 0.254 to 0.059; the video Platt fit lowered pooled ECE on a separate 161-video test split from 0.161 to 0.078. The temperature-scaled values were measured on the fitting scores and are therefore optimistic.

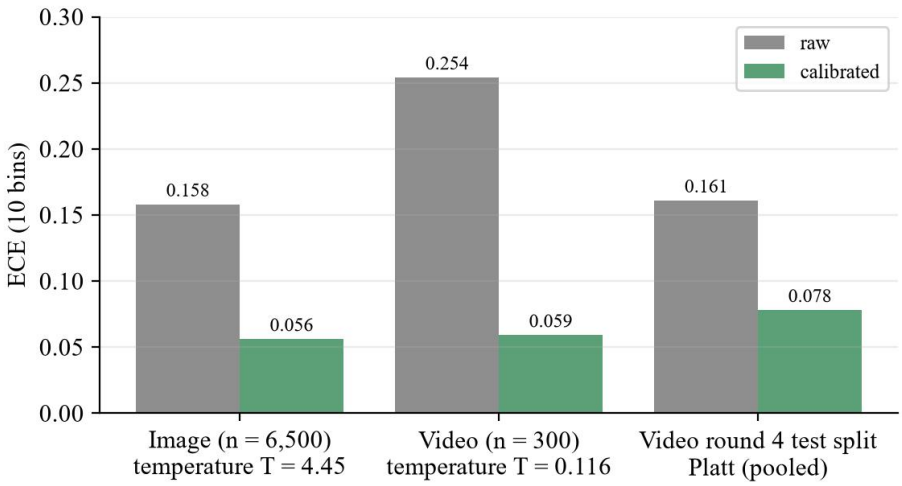


Figure 5.17: Expected calibration error before and after calibration

5.2.8 Fairness Across Demographic Groups

To check whether genuine photos of some groups are wrongly flagged more often, 600 real UTKFace portraits were scored, 60 per gender-race group (Figure 5.18). The deployed ensemble's specificity is 94.8% overall but ranges from 90.0% to 100% across groups; ConvNeXt alone spans 96.7–100% and CLIP alone 91.7–100%. CLIP's two weakest groups are exactly the ensemble's two weakest, pointing again to CLIP as the source of false alarms. With 60 images per group the ranking of specific groups is noisy; the size of the spread and the sub-model it comes from are the reliable findings.

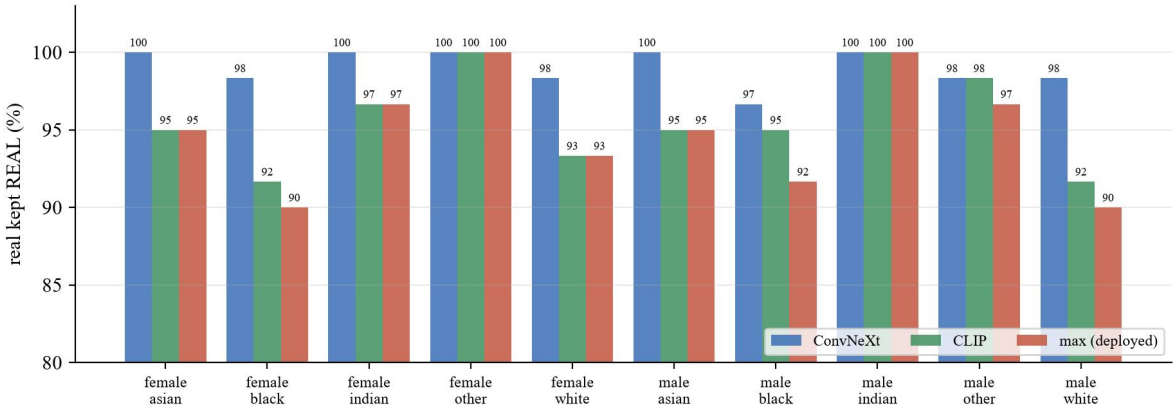


Figure 5.18: Real-photo specificity per gender and race group (600 UTKFace portraits)

5.2.9 Identity Verification

On 1,000 LFW DevTest pairs scored with the deployed pipeline, same-person similarity averaged 0.720 (σ 0.201) and different-person similarity 0.029 (σ 0.160), with ROC-AUC 0.974. At the chosen threshold of 0.44 the accuracy is 96.5% with a 0.4% false-accept rate and a 6.6% false-reject rate (Figures 5.19 and 5.20). LFW consists of frontal, well-lit

press photographs; webcam enrolment under poor lighting will plausibly overlap more, so this is a data-grounded starting point, not a guaranteed production error rate.

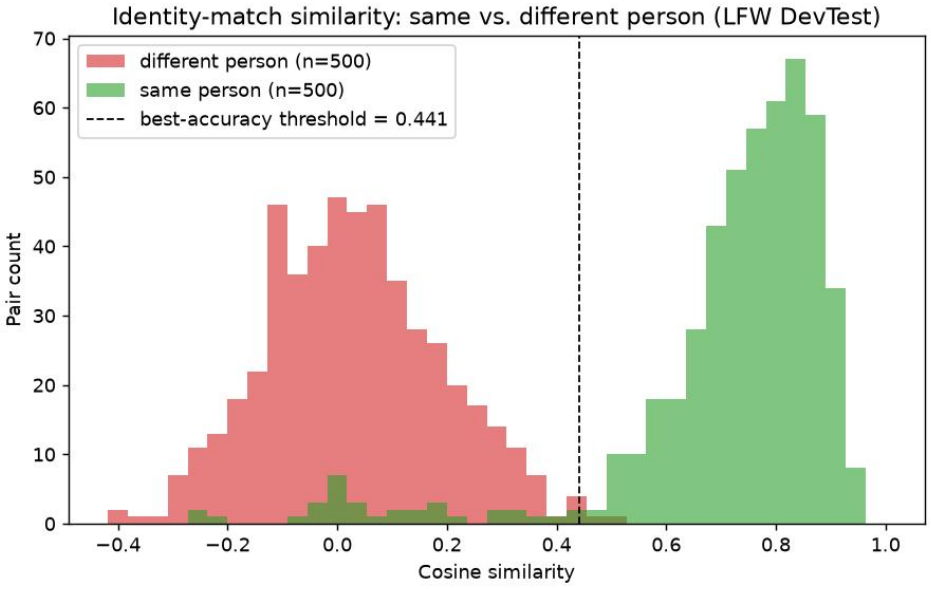


Figure 5.19: Distribution of cosine similarity for same-person and different-person LFW pairs

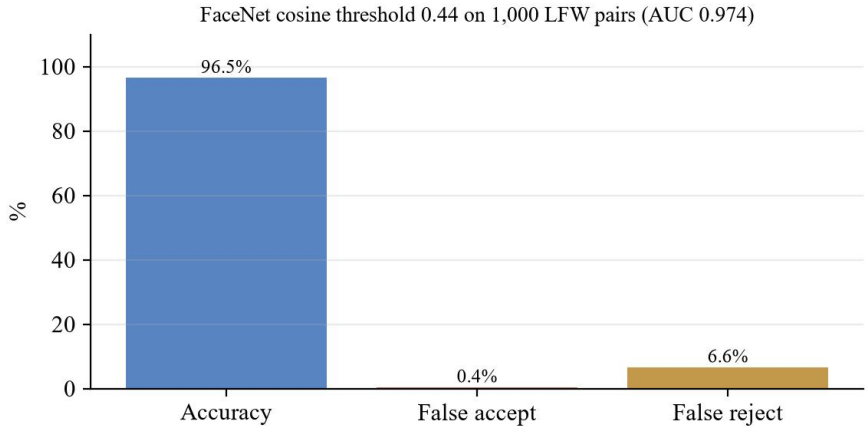


Figure 5.20: Identity matching at threshold 0.44 on LFW

5.2.10 The Working System

Figures 5.21 to 5.30 show the deployed application processing real files through the live API on CPU, captured on the final build. The home page (Figure 5.21) introduces the examination flow; the verify page (Figure 5.22) accepts all three media types; history and dashboard (Figures 5.23 and 5.24) show only the signed-in user's scans.

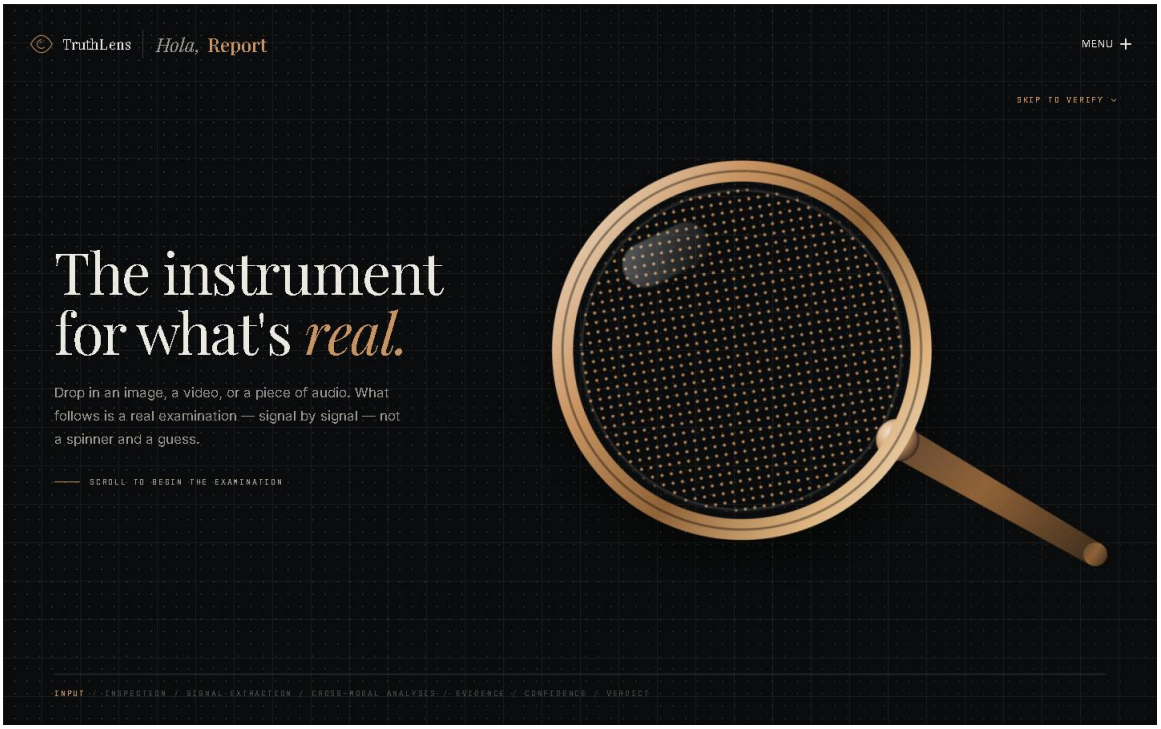


Figure 5.21: Home page

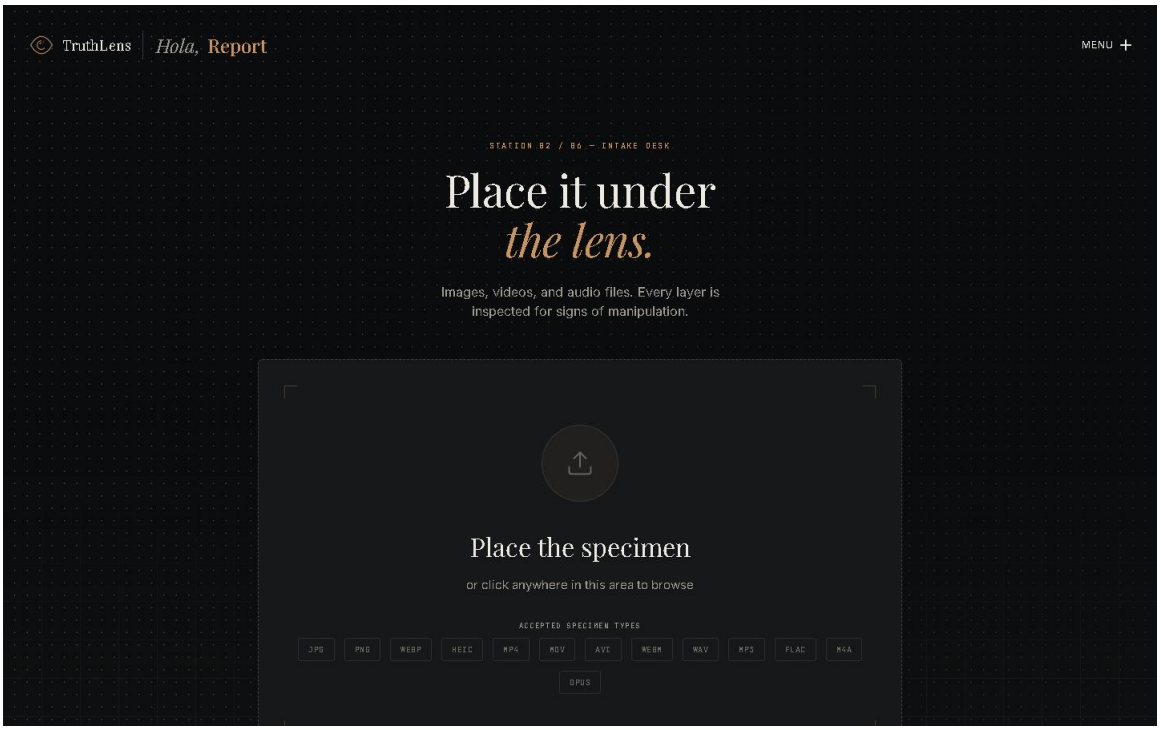


Figure 5.22: Verify page: upload of an image, video or audio file

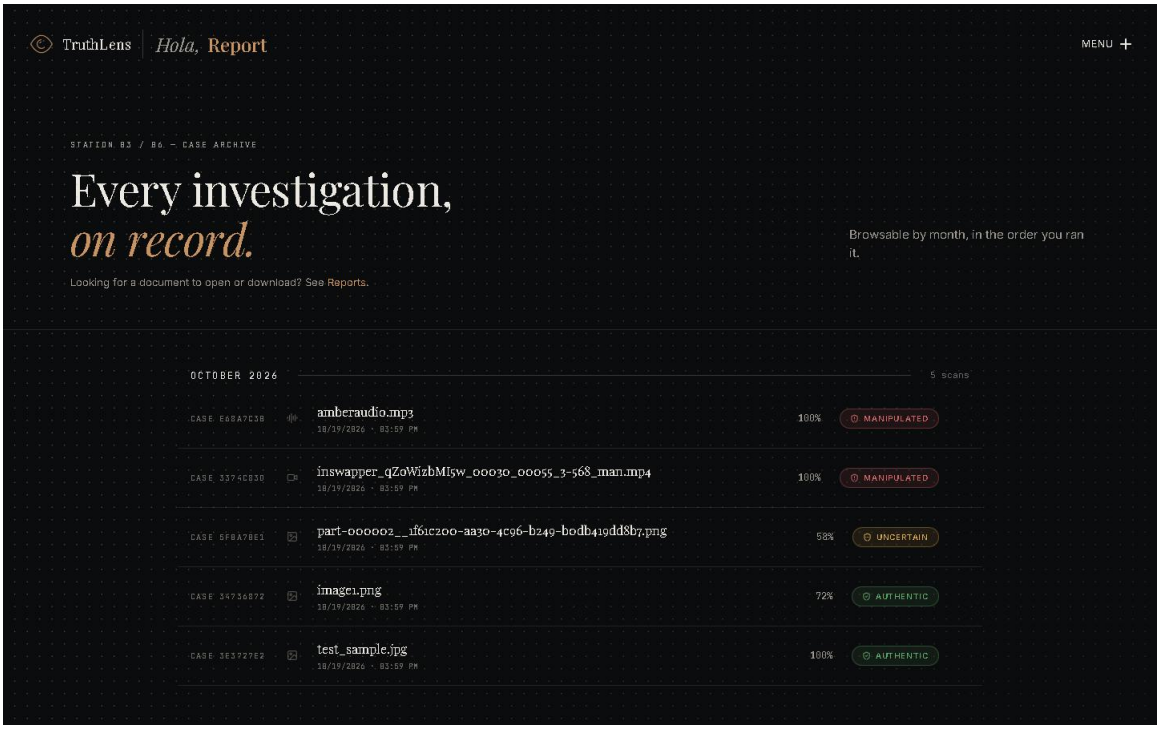


Figure 5.23: Scan history (owner-only)

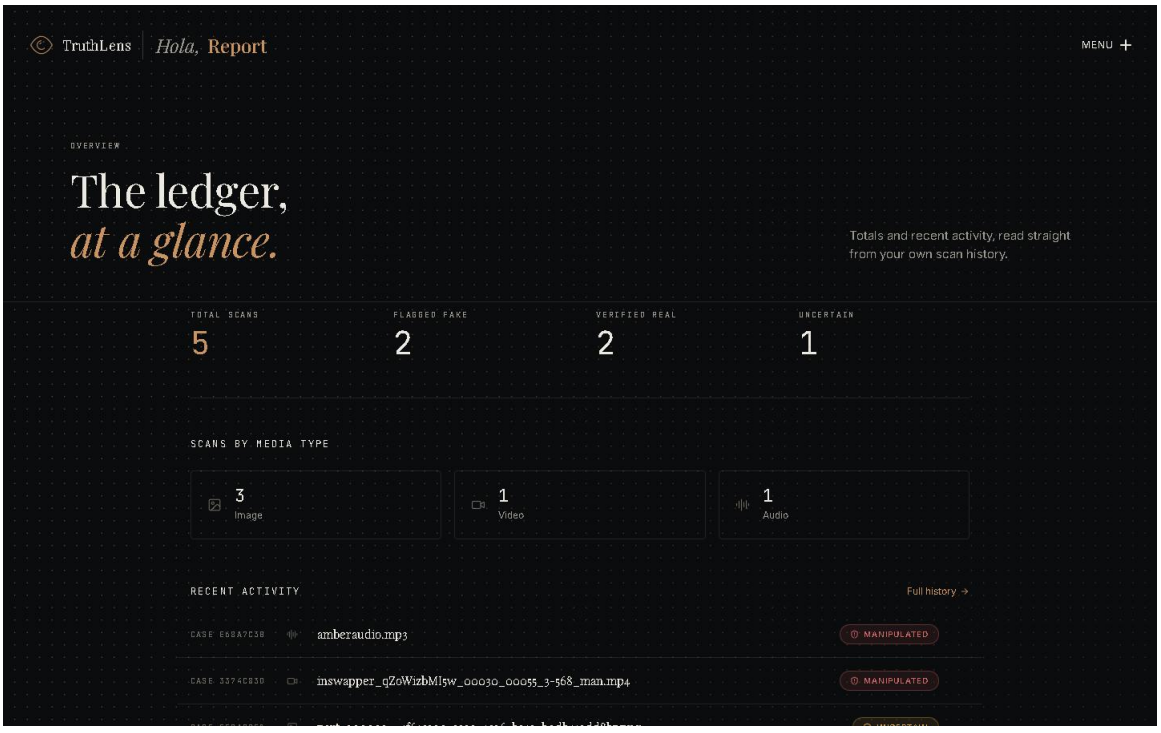


Figure 5.24: Dashboard built from real counts only

Figure 5.25 shows a photorealistic ChatGPT image, one of the documented blind spots. The models call it REAL, but its C2PA credentials declare AI generation, so the interface shows the provenance conflict and switches the headline to the warning style; Grad-CAM is faded because ConvNeXt's own fake probability is low and the check that drove the score has no map. Figure 5.26 shows a portrait-app image whose stored verdict is

UNCERTAIN (58%); its embedded metadata declares AI generation, so the same conflict display applies.

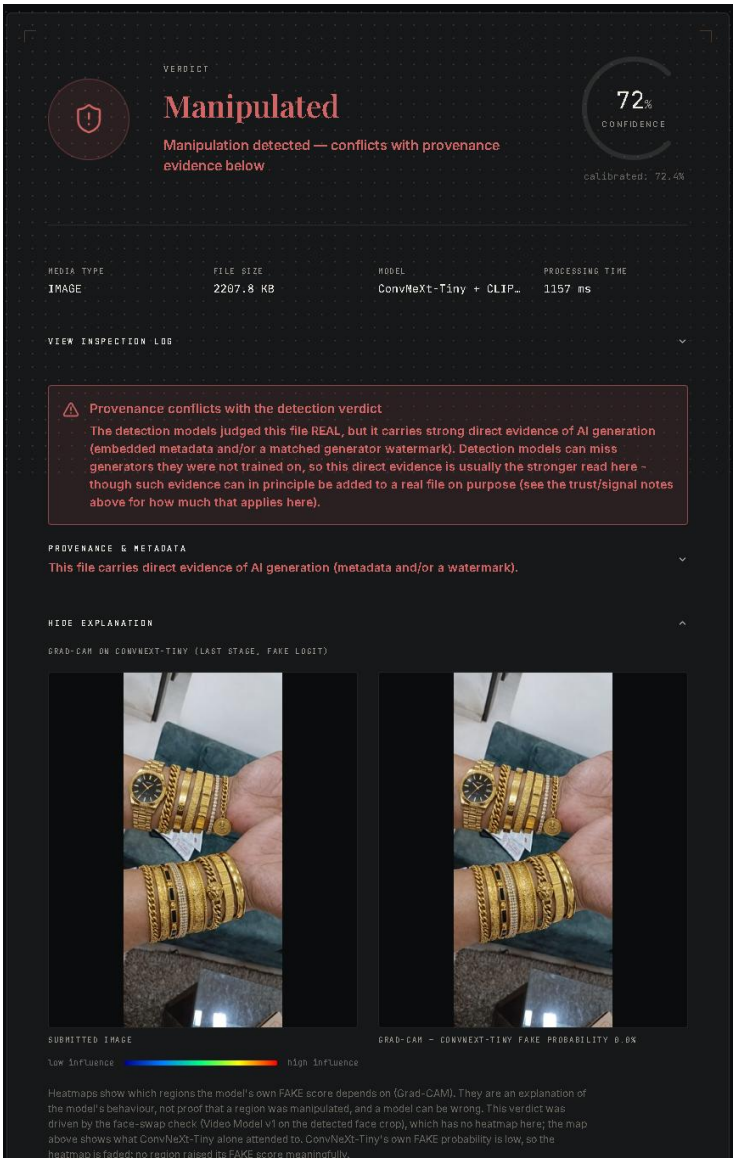


Figure 5.25: Result for a ChatGPT-generated photograph: REAL by the models, AI-declared by provenance



Figure 5.26: Result for a portrait-app image: UNCERTAIN by the models, AI-declared by metadata

Figure 5.27 shows a face-swap video (an RTFS inswapper clip; RTFS was part of round-4 training, so this is a demonstration, not evidence of generalisation): the calibrated probability is 99.8%, sixteen of sixteen frames had a detected face, and the per-frame bars, the exact crops and Grad-CAM on the three most suspicious frames are shown. Figure 5.28 shows an audio scan with the per-window score and the time-saliency curve over the spectrogram; the clip's ground truth is not known, so it illustrates the output only.



Figure 5.27: Video result: per-frame scores, face crops and Grad-CAM on the most suspicious frames

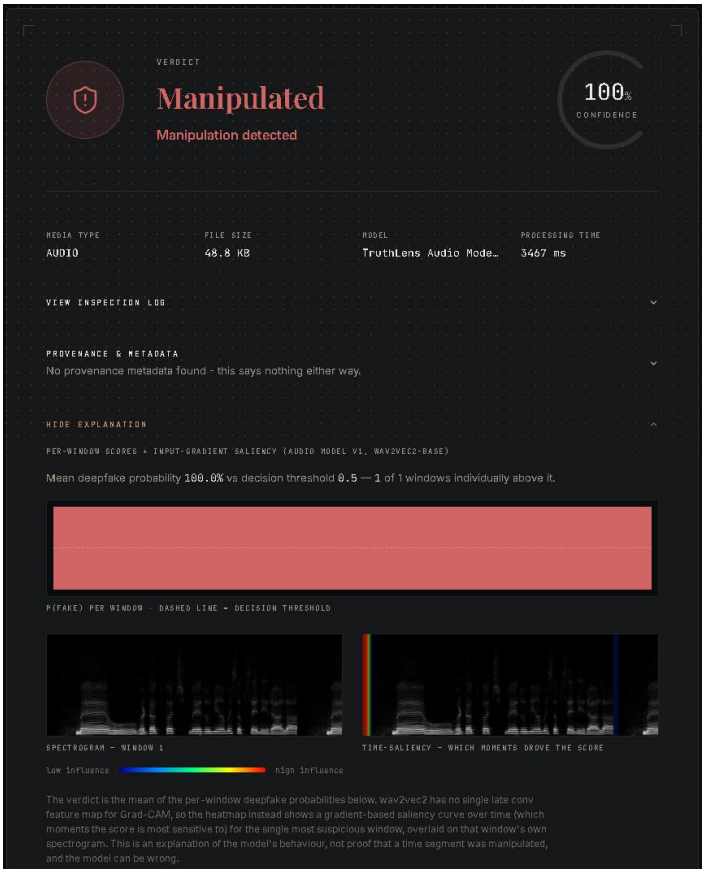


Figure 5.28: Audio result: per-window score and input-gradient saliency over the spectrogram

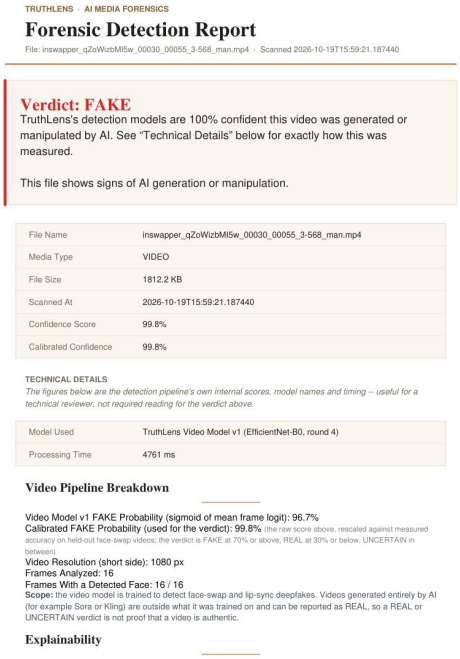


Figure 5.29: First page of the generated PDF forensic report for the video

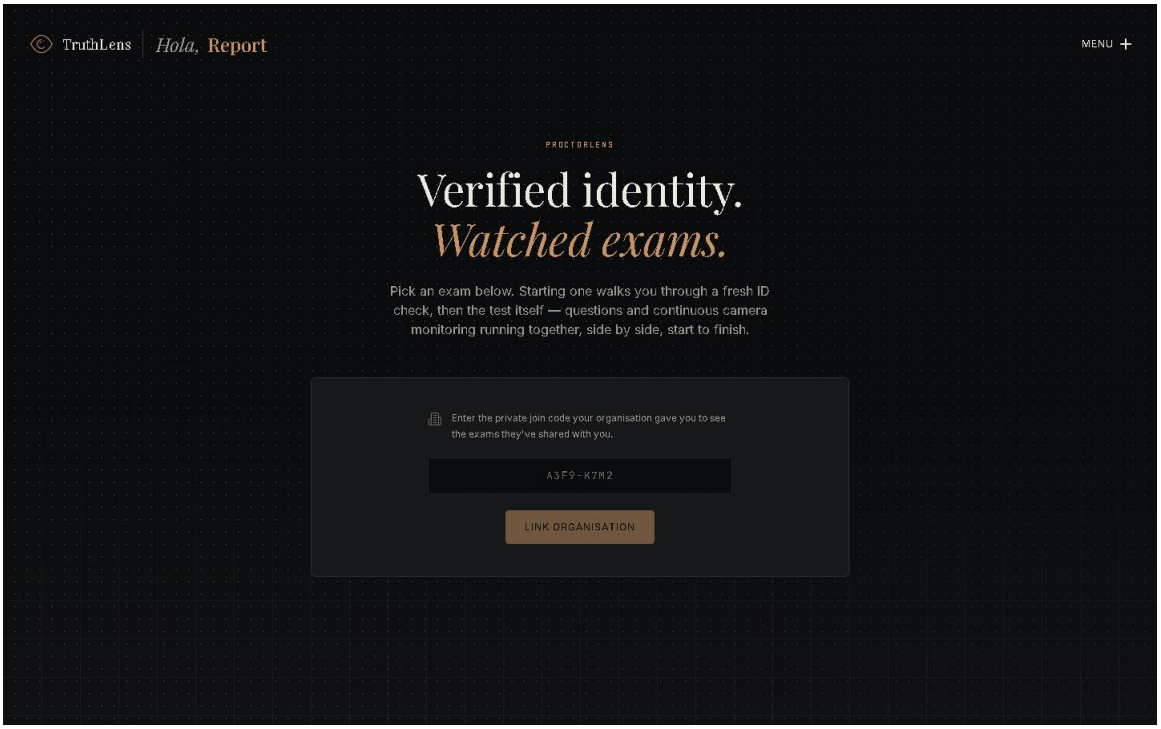


Figure 5.30: ProctorLens student hub: joining an organisation to see its exams

5.3 Comparison with Existing Methods

Published detectors are evaluated on different data, so their headline numbers cannot be compared directly with ours. Table 5.2 therefore compares the configurations we built and measured on the same 6,532 images (all sources pooled, threshold 0.5), together with the video model on 310 unseen clips. The deployed maximum rule has the best pooled accuracy, recall and F1, at the cost of precision; ConvNeXt alone is the most precise.

Table 5.2: Performance comparison of the evaluated methods

Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	AUC
ConvNeXt-Tiny (end-to-end, cf. [8])	80.2	98.6	63.1	77.0	0.901
CLIP ViT-B/16 probe (cf. [9])	79.1	91.9	66.0	76.8	0.857
Mean of ConvNeXt and CLIP	79.8	97.6	63.0	76.6	0.888
Proposed: max ensemble (deployed)	80.9	91.6	70.0	79.4	0.886
Video Model v1, epoch 11 (310 unseen clips)	79.4	82.7	74.2	78.2	0.892

Table 5.3 compares TruthLens with typical published detectors and public online checkers by capability. The contribution is less a new network than a complete, measured system: three modalities, an evidence layer, explicit uncertainty, honest evaluation and a proctoring application of the same components.

Table 5.3: Capability comparison with typical existing approaches

Capability	Research detectors [2, 9, 16]	Typical online checkers	TruthLens
Image, video and audio	One modality each	Often one	All three
Explanation of each decision	Rare	Rare	Grad-CAM, per-frame, saliency
Provenance (C2PA, metadata, watermark)	No	Sometimes	Yes, with conflict note
Explicit UNCERTAIN band	No	No	Yes
Calibrated confidence	Rarely reported	Not stated	Temperature / Platt
Seen vs unseen evaluation	Partly	Not published	Yes, every source tagged
Robustness and fairness measured	Some	Not published	Yes
Identity verification and proctoring	No	No	ProctorLens

5.4 Discussion and Limitations

The results support one main lesson: a detector that scores 98% on benchmark-like data can be close to chance on the next generator, and a user cannot tell which situation applies to their file. For that reason TruthLens describes REAL as "no evidence of generation found" rather than "authentic", reserves FAKE for clear scores, and places provenance and explanations beside the number. The main limitations are:

- **Training-data coverage.** Photorealistic GPT-class images and some consumer face-swap apps are absent from the training sources, and both are confirmed blind spots; fully generated video is out of scope.
- **Dataset shortcuts.** DeepDetect's compression history, GenImage's JPEG-versus-PNG split and RTFS's different encodes may inflate measured accuracy.
- **Small samples.** The blind-spot sets (8 and 15 images), the team videos (10), the video robustness subset (30) and the per-group fairness samples (60) support qualitative conclusions only.
- **Not re-measured for the newest models.** The robustness study and the 300-clip RTFS harness used the epoch-11 video weights; the audio model has not been evaluated in the harness.

- **Proctoring thresholds.** Apart from the identity threshold, monitoring thresholds are placeholders without a labelled validation set, so flags are prompts for human review.
- **Engineering.** Job state lives in one server process, and inference is CPU-only in our deployment.

A forensic tool can cause harm in both directions: a false FAKE can discredit a genuine photograph and a false REAL can lend credibility to a hoax. The verdict bands, explicit conflict notes, "lead, not proof" wording and the absence of any automated action on a verdict (other than exam termination by a limit the organisation sets) are our mitigations.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

This project designed, built and evaluated TruthLens, a multimodal platform that detects AI-generated and manipulated images, videos and speech and presents each verdict together with the evidence behind it. Each media type has its own pipeline: a ConvNeXt-Tiny, CLIP and face-swap ensemble for images; a face-crop EfficientNet-B0 with Platt calibration and a resolution guard for video; and a fine-tuned wav2vec 2.0 model for audio. Around them, an evidence layer adds Grad-CAM and saliency explanations, C2PA, metadata and watermark provenance with explicit conflict notes, calibrated confidence and PDF reports. ProctorLens extends the same components to identity-verified, continuously monitored online examinations with server-side enforcement.

All the objectives set in Chapter 1 were met. The system is a complete, tested and deployable application with owner-only privacy, upload validation, rate limiting, 530 passing automated tests and a production configuration. The evaluation was as important as the implementation: it showed near-perfect accuracy on familiar data (98.4%) and weak generalisation to unseen generators (53.2%), a severe video fragility under compression that guided the round-4 fine-tune (held-out AUC 0.765 to 0.961), the value of provenance exactly where detectors fail, and a CLIP-driven spread in false alarms across demographic groups. The central conclusion is that detector scores should be presented as one strand of evidence, with their uncertainty and limits stated, rather than as a final answer.

6.2 Future Scope

- Retrain the image models with more diverse generators (GPT-class image models, Flux, recent Midjourney versions) and with noise and compression augmentation, then revisit the maximum-versus-mean trade-off.

- Source fake clips from consumer face-swap apps and select checkpoints on a real-world set that is not also used for selection; repeat the robustness study for the round-4 video model.
- Evaluate the audio model in the harness on a labelled dataset and under MP3, noise and volume degradation, and fit an audio calibration.
- Add a temporal or audio-visual model for lip-sync consistency in talking-head forgeries, and coverage of fully generated video.
- Fit calibration on held-out splits and extend the fairness analysis to video, audio and identity matching.
- Validate the proctoring thresholds on labelled webcam data and add a trained anti-spoofing (liveness) model.
- Move job state to a shared queue for horizontal scaling and add GPU inference for faster scans.

Chapter 7

List of Publications

7.1 List of Published Papers

1. M. Jaiswal, R. Shinde, A. Pethkar, and S. Thakur, "TruthLens: An Evidence-Oriented Multimodal Platform for Detecting AI-Generated Media," manuscript prepared in IEEE conference format, 2026. [Venue, status and DOI to be filled in by the authors.]

7.2 Details of Participation in Project Competitions

[To be filled in by the team: name of the competition, organiser, date and outcome, if any.]

References

[1] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 10684–10695.

[2] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, “FaceForensics++: Learning to detect manipulated facial images,” in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 1–11.

[3] X. Wang et al., “ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech,” Comput. Speech Lang., vol. 64, Art. no. 101114, 2020.

[4] Y. Li, X. Yang, P. Sun, H. Qi, and S. Lyu, “Celeb-DF: A large-scale challenging dataset for DeepFake forensics,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 3207–3216.

[5] J. J. Bird and A. Lotfi, “CIFAKE: Image classification and explainable identification of AI-generated synthetic images,” IEEE Access, vol. 12, pp. 15642–15650, 2024.

[6] M. Zhu et al., “GenImage: A million-scale benchmark for detecting AI-generated image,” in Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track, 2023.

[7] Coalition for Content Provenance and Authenticity, “C2PA technical specification,” version 2.1, 2024. [Online]. Available: https://c2pa.org/specifications/

[8] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, “CNN-generated images are surprisingly easy to spot... for now,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 8695–8704.

[9] U. Ojha, Y. Li, and Y. J. Lee, “Towards universal fake image detectors that generalize across generative models,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023, pp. 24480–24489.

[10] J. Frank, T. Eisenhofer, L. Schönherr, A. Fischer, D. Kolossa, and T. Holz, “Leveraging frequency analysis for deep fake image recognition,” in Proc. Int. Conf. Mach. Learn. (ICML), 2020, pp. 3247–3258.

[11] A. Radford et al., “Learning transferable visual models from natural language supervision,” in Proc. Int. Conf. Mach. Learn. (ICML), 2021, pp. 8748–8763.

[12] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 11976–11986.

[13] B. Dolhansky et al., “The DeepFake Detection Challenge (DFDC) dataset,” arXiv preprint arXiv:2006.07397, 2020.

[14] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in Proc. Int. Conf. Mach. Learn. (ICML), 2019, pp. 6105–6114.

[15] B. Zi, M. Chang, J. Chen, X. Ma, and Y.-G. Jiang, “WildDeepfake: A challenging real-world dataset for deepfake detection,” in Proc. 28th ACM Int. Conf. Multimedia, 2020, pp. 2382–2390.

[16] J.-w. Jung et al., “AASIST: Audio anti-spoofing using integrated spectro-temporal graph attention networks,” in Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP), 2022, pp. 6367–6371.

[17] H. Tak, M. Todisco, X. Wang, J.-w. Jung, J. Yamagishi, and N. Evans, “Automatic speaker verification spoofing and deepfake detection using wav2vec 2.0 and data augmentation,” in Proc. Odyssey: The Speaker and Language Recognition Workshop, 2022, pp. 112–119.

[18] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 12449–12460.

[19] N. M. Müller, P. Czempin, F. Dieckmann, A. Froghyar, and K. Böttinger, “Does audio deepfake detection generalize?” in Proc. Interspeech, 2022, pp. 2783–2787.

[20] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 618–626.

[21] K. Simonyan, A. Vedaldi, and A. Zisserman, “Deep inside convolutional networks: Visualising image classification models and saliency maps,” in Proc. Int. Conf. Learn. Represent. (ICLR) Workshop, 2014.

[22] ShieldMnt, “invisible-watermark: Python library for invisible image watermarking,” GitHub repository. [Online]. Available: https://github.com/ShieldMnt/invisible-watermark

[23] N. Krawetz, “A picture's worth: Digital image analysis and forensics,” presented at Black Hat Briefings USA, 2007.

[24] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in Proc. Int. Conf. Mach. Learn. (ICML), 2017, pp. 1321–1330.

[25] J. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” in Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61–74.

[26] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, “Joint face detection and alignment using multitask cascaded convolutional networks,” IEEE Signal Processing Letters, vol. 23, no. 10, pp. 1499–1503, 2016.

[27] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A unified embedding for face recognition and clustering,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2015, pp. 815–823.

[28] Q. Cao, L. Shen, W. Xie, O. M. Parkhi, and A. Zisserman, “VGGFace2: A dataset for recognising faces across pose and age,” in Proc. IEEE Int. Conf. Automatic Face & Gesture Recognition (FG), 2018, pp. 67–74.

[29] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, “Labeled Faces in the Wild: A database for studying face recognition in unconstrained environments,” Univ. of Massachusetts, Amherst, Tech. Rep. 07-49, 2007.

[30] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” software, version 8.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[31] Z. J. Wang, E. Montoya, D. Munechika, H. Yang, B. Hoover, and D. H. Chau, “DiffusionDB: A large-scale prompt gallery dataset for text-to-image generative models,” in Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL), 2023, pp. 893–911.

[32] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR), 2001, pp. I-511–I-518.

[33] J. J. Bird and A. Lotfi, “Real-time detection of AI-generated speech for DeepFake voice conversion,” arXiv preprint arXiv:2308.12734, 2023.

[34] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An ASR corpus based on public domain audio books,” in Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP), 2015, pp. 5206–5210.

[35] Y. Zhao et al., “Voice Conversion Challenge 2020: Intra-lingual semi-parallel and cross-lingual voice conversion,” in Proc. Joint Workshop Blizzard Challenge and Voice Conversion Challenge, 2020, pp. 80–98.

[36] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2019.

[37] Z. Zhang, Y. Song, and H. Qi, “Age progression/regression by conditional adversarial autoencoder,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 5810–5818.

Appendix A: Installation and User Guide

This appendix explains how to install and run the system from the project repository and how to use it. The commands are the ones documented in the repository (README.md, CLAUDE.md and docs/DEPLOYMENT.md).

A.1 Prerequisites

- Python 3.11 with the packages in backend/requirements.txt (PyTorch CPU build, FastAPI, OpenCLIP, Transformers, facenet-pytorch, ONNX Runtime and others) installed in a virtual environment.
- Node.js 20 and npm for the React frontend.
- Optionally Docker Desktop (Windows) or Docker Engine with Compose (Linux) for containerised runs.
- The model checkpoint files listed in models/CHECKSUMS.sha256. They are too large for git and are distributed separately; the CLIP and wav2vec 2.0 backbones download automatically on first use.

A.2 Obtaining and verifying the checkpoints

If the checkpoints are hosted in a folder you control, the fetch script downloads each one and verifies its SHA-256 checksum; a file with a wrong checksum is discarded rather than kept. The --check option only verifies files that are already in place.

```
python backend/scripts/fetch_checkpoints.py --base-url https://YOUR-HOST/truthlens-
models/
python backend/scripts/fetch_checkpoints.py --check
```

A.3 Running locally without Docker

Start the backend from the backend folder, pointing it at the image checkpoints (Windows PowerShell shown; the other checkpoints are found at their default paths):

```
cd backend
$env:IMAGE_MODEL_CHECKPOINT="../models/checkpoints/image/convnext_tiny_diversified_v2.p
th"
$env:CLIP_MODEL_CHECKPOINT="../models/checkpoints/image_clip/clip_head_round3_portrait_
app.pth"
$env:IMAGE_MODEL_DEVICE="cpu"; $env:CLIP_MODEL_DEVICE="cpu"
```

```
../.venv/Scripts/python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Then start the frontend in a second terminal. The development server runs at http://localhost:3000 and forwards /api requests to the backend; the interactive API documentation is at http://localhost:8000/docs.

```
cd frontend
npm install
npm run dev
```

A.4 Running with Docker

For development both services start with one command. For production, copy the example environment file, set a long random signing key (the server refuses to start with the default one) and use the production compose file, which serves the site through nginx with security headers and rate limits.

```
docker-compose up --build
cp .env.example .env
python -c "import secrets; print(secrets.token_urlsafe(48))"   # paste as
AUTH_SECRET_KEY in .env
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml ps
```

A.5 Running the tests

The full suite needs the checkpoints; the second command is the subset that CI runs without any model files.

```
cd backend
../.venv/Scripts/python.exe -m pytest
../.venv/Scripts/python.exe -m pytest -m "not models"
```

A.6 Verifying a file with TruthLens

1. Sign up with a name, email, password (at least 8 characters) and account type, or log in (Figure C.1).
2. Open Verify and choose an image, video or audio file of up to 100 MB. Images and audio are scanned immediately; a video scan shows a progress bar with its stage and can be cancelled.

3. Read the result card: the verdict (Authentic, Uncertain or Manipulated), the confidence and the calibrated confidence, the model used and the processing time (Figure C.2).
4. Open Provenance and Metadata to see Content Credentials, embedded metadata and watermark results; a red conflict note appears when the file declares itself AI-generated but the models did not flag it.
5. Open the explanation to see the Grad-CAM heat-map (image), per-frame scores and crops (video) or per-window scores and the saliency curve (audio).
6. Answer "Was this result correct?" if you wish, and choose Download PDF Report for a shareable forensic report (Figures C.3 and C.4).
7. Find every past scan under History and Reports, and the totals under Dashboard. Deleting your account from the account menu removes all of your files, results and reports.

A.7 Using ProctorLens

Organisation account. Sign up with the organisation account type and open the Admin page. The Setup tab shows the private join code to give to students (it can be regenerated) and the roster; the Exams tab creates an exam with its pass mark and flag limit and adds multiple-choice questions with their correct answers; the Activity tab lists attempts with their status, score and flags, and each session report can be downloaded as a PDF.

Student account. Open ProctorLens, enter the organisation's join code and pick an exam. Before every attempt the student gives consent, photographs a government ID and takes a live selfie; the attempt starts only if the two faces match. During the exam the camera stays on and periodic checks run in the background; if the number of flagged checks reaches the exam's limit, the server ends the attempt. Otherwise the attempt is scored when the student submits it.

Appendix B: Selected Source Code

The listings below are copied verbatim from the repository by the report build script, so they always match the submitted code. They show the parts of the system on which the verdicts depend; the complete source, tests and evaluation scripts are in the project repository.

Listing B.1: Image ensemble: maximum of ConvNeXt, CLIP and the face-swap check *(backend/app/services/image/detector.py)*

```
convnext_fake_probability = convnext_prediction["fake_probability"]
clip_fake_probability = clip_prediction["fake_probability"]


fake_probability = max(
    convnext_fake_probability, clip_fake_probability
)

# Face-swap check (CLAUDE.md section 28). The whole-image models above are near chance on
face swaps (a swap is
# a real photo with one face replaced); Video Model v1 round 4, run on the detected face
crop, is not. Applies
# only when a frontal face >= 64 px is found; its Platt-calibrated P(FAKE) joins the same
max rule. A failure
# here must never break the scan, so it just means "check not applied".
face_swap_probability, face_swap_face_px = _face_swap_check(path)
ensemble_fake_probability = fake_probability
if face_swap_probability is not None:
    fake_probability = max(fake_probability, face_swap_probability)
real_probability = 1 - fake_probability


# Verdict banding: +-0.2 margin around FAKE_THRESHOLD, same convention
# already used by the audio and video detectors (services/audio/
# detector.py, services/video/detector.py), so a genuinely close call
# (including disagreement between the two models above) comes back
# UNCERTAIN instead of a forced argmax.
verdict = _verdict(fake_probability)
```

Listing B.2: Verdict banding around the threshold *(backend/app/services/image/detector.py)*

```
def _verdict(fake_probability: float) -> str:
    if fake_probability >= settings.FAKE_THRESHOLD + 0.2:
        return "FAKE"
    if fake_probability <= settings.FAKE_THRESHOLD - 0.2:
        return "REAL"
    return "UNCERTAIN"
```

Listing B.3: Temperature and Platt calibration *(backend/app/services/calibration.py)*

```
def calibrate(probability: float, media_type: str) -> float | None:
    """Returns the calibrated probability for `media_type`, or None if nothing has been
fit for it
```

```
    (e.g. audio, or a media type with no eval data yet).

    Two forms: temperature scaling (`{"temperature": T}`, sigmoid(logit(p) / T), image)
and Platt scaling
    (`{"method": "platt", "a": a, "b": b}`, sigmoid(a * logit(p) + b), video since 2026-
10-10 - the bias term is
    needed because Video Model v1's raw scores are both compressed toward 0.5 and centred
on 0.525, not 0.5)."""
    cfg = _load().get(media_type)
    if cfg is None:
        return None
    p = min(max(probability, _EPS), 1 - _EPS)
    z = math.log(p / (1 - p))
    if cfg.get("method") == "platt":
        return 1.0 / (1.0 + math.exp(-(cfg["a"] * z + cfg["b"])))
    return 1.0 / (1.0 + math.exp(-z / cfg["temperature"]))
```

Listing B.4: Audio windowing (up to six 4-second windows)

(backend/app/services/audio/model_v1/optimized.py)

```
def _build_windows(samples: np.ndarray) -> np.ndarray:
    n = samples.shape[0]
    if n <= 0:
        raise ValueError("Empty audio signal.")

    if n <= common.WINDOW_SAMPLES:
        return _tile_pad(samples, common.WINDOW_SAMPLES)[None, :]

    n_windows = min(common.MAX_WINDOWS, int(np.ceil(n / common.WINDOW_SAMPLES)))
    starts = np.linspace(0, max(n - common.WINDOW_SAMPLES, 0), n_windows).astype(int)
    windows = np.stack([samples[s:s + common.WINDOW_SAMPLES] for s in starts])
    if windows.shape[1] < common.WINDOW_SAMPLES:
        windows = np.stack([_tile_pad(w, common.WINDOW_SAMPLES) for w in windows])
    return windows
```

Listing B.5: Cosine similarity of face embeddings *(backend/app/services/identity/detector.py)*

```
def _cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = sum(x * x for x in a) ** 0.5
    norm_b = sum(y * y for y in b) ** 0.5
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return dot / (norm_a * norm_b)
```

Listing B.6: Server-side termination of an exam attempt at the flag limit

(backend/app/services/exams/enforcement.py)

```
async def enforce_flag_threshold(db: AsyncSession, session_id: str) -> None:
    attempt = (
        await db.execute(
            select(ExamAttempt)
            .where(ExamAttempt.session_id == session_id)
            .where(ExamAttempt.status == "in_progress")
        )
    ).scalar_one_or_none()
    if attempt is None:
```

```
        return  # not an exam session, or already terminal -- nothing to enforce

    exam = (await db.execute(select(Exam).where(Exam.exam_id ==
attempt.exam_id))).scalar_one()

    flagged_count = (
        await db.execute(
            select(func.count())
            .select_from(MonitoringCheck)
            .where(MonitoringCheck.session_id == session_id)
            .where(MonitoringCheck.flagged == True)  # noqa: E712 (SQLAlchemy boolean
comparison)
        )
    ).scalar_one()

    if flagged_count < exam.max_flags:
        return

    now = datetime.utcnow().isoformat()
    attempt.status = "terminated_for_flags"
    attempt.ended_at = now

    session = (
        await db.execute(select(MonitoringSession).where(MonitoringSession.session_id ==
session_id))
    ).scalar_one_or_none()
    if session is not None and session.ended_at is None:
        session.ended_at = now

    await db.flush()
```

Appendix C: Additional Screenshots and Sample Report

These screenshots were captured from the final build processing real files through the live API, in the same session as those in Section 5.2. The sample report pages are the PDF generated by the system for the ChatGPT photograph of Figure 5.25, and the second page of the video report.

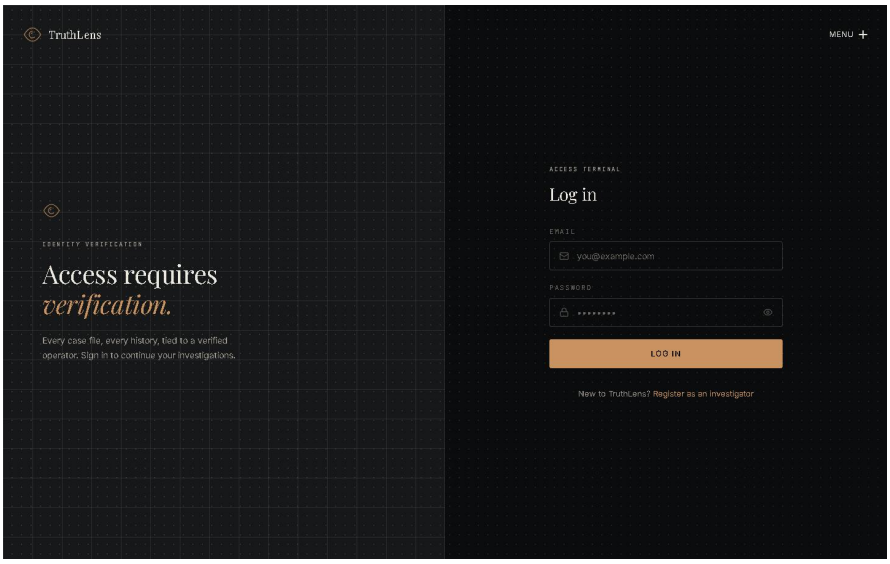


Figure C.1: Login page

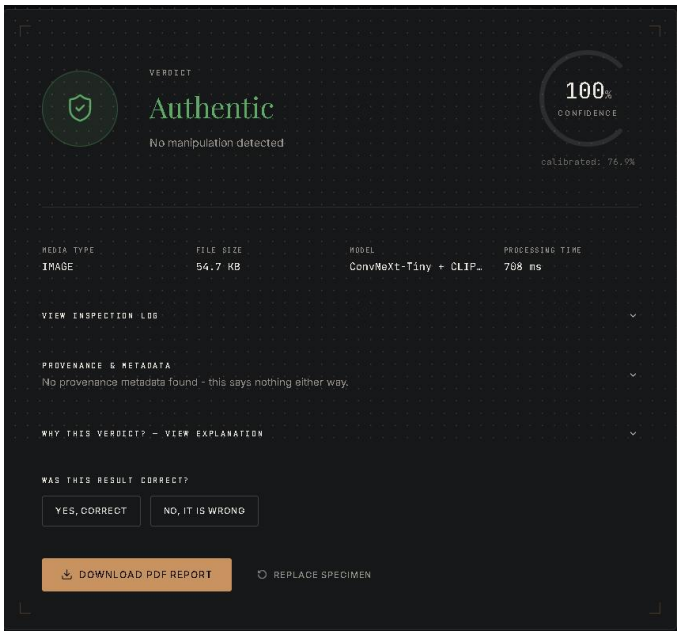
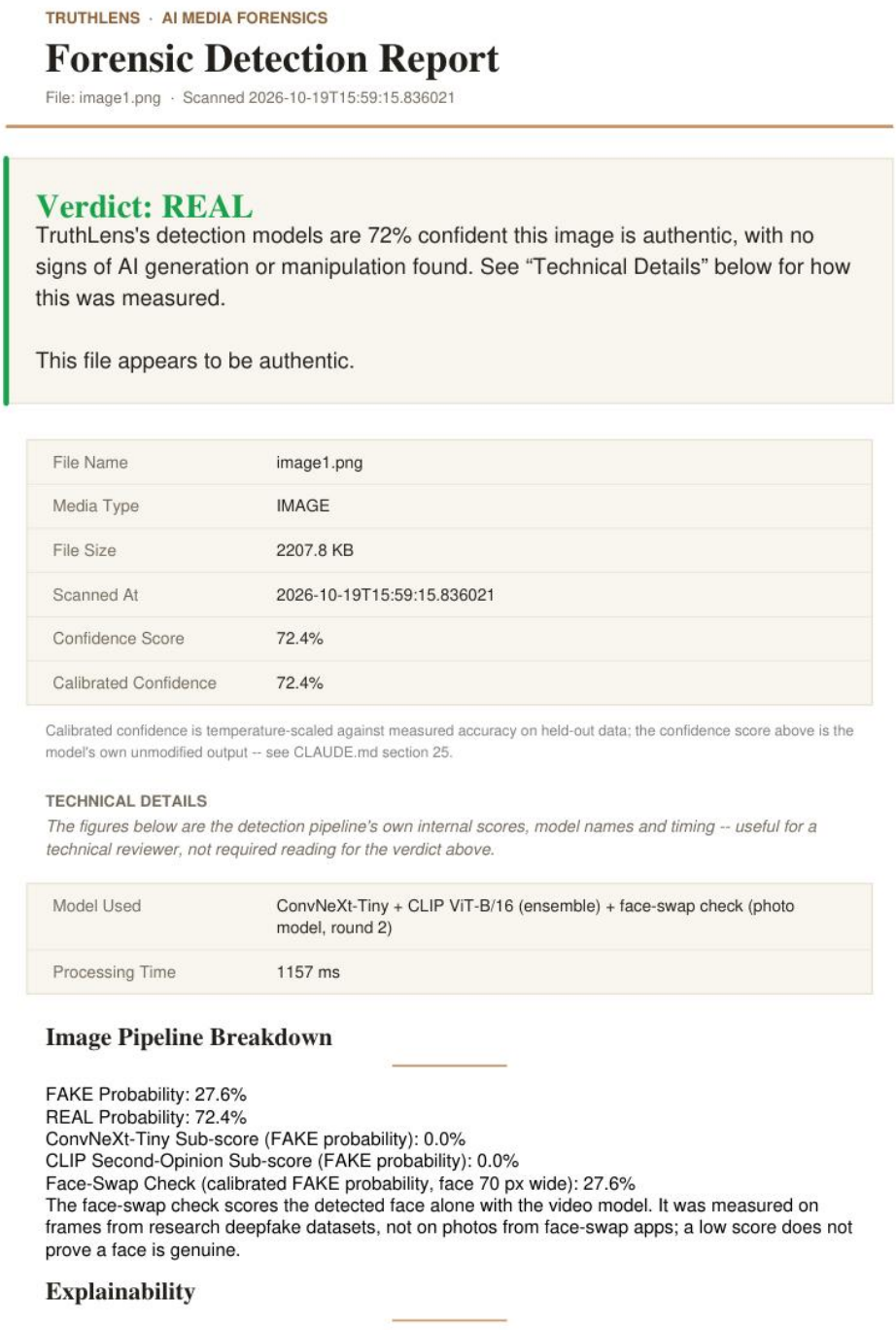


Figure C.2: Result card for a genuine photograph: Authentic, no provenance metadata, explanation collapsed



TRUTHLENS · AI MEDIA FORENSICS

Forensic Detection Report

File: image1.png · Scanned 2026-10-19T15:59:15.836021

Verdict: REAL

TruthLens's detection models are 72% confident this image is authentic, with no signs of AI generation or manipulation found. See “Technical Details” below for how this was measured.

This file appears to be authentic.

File Name	image1.png
Media Type	IMAGE
File Size	2207.8 KB
Scanned At	2026-10-19T15:59:15.836021
Confidence Score	72.4%
Calibrated Confidence	72.4%

Calibrated confidence is temperature-scaled against measured accuracy on held-out data; the confidence score above is the model's own unmodified output -- see CLAUDE.md section 25.

TECHNICAL DETAILS

The figures below are the detection pipeline's own internal scores, model names and timing -- useful for a technical reviewer, not required reading for the verdict above.

Model Used	ConvNeXt-Tiny + CLIP ViT-B/16 (ensemble) + face-swap check (photo model, round 2)
Processing Time	1157 ms

Image Pipeline Breakdown

FAKE Probability: 27.6%
REAL Probability: 72.4%
ConvNeXt-Tiny Sub-score (FAKE probability): 0.0%
CLIP Second-Opinion Sub-score (FAKE probability): 0.0%
Face-Swap Check (calibrated FAKE probability, face 70 px wide): 27.6%
The face-swap check scores the detected face alone with the video model. It was measured on frames from research deepfake datasets, not on photos from face-swap apps; a low score does not prove a face is genuine.

Explainability

Figure C.3: Generated PDF report for the ChatGPT photograph, page 1: verdict and technical details

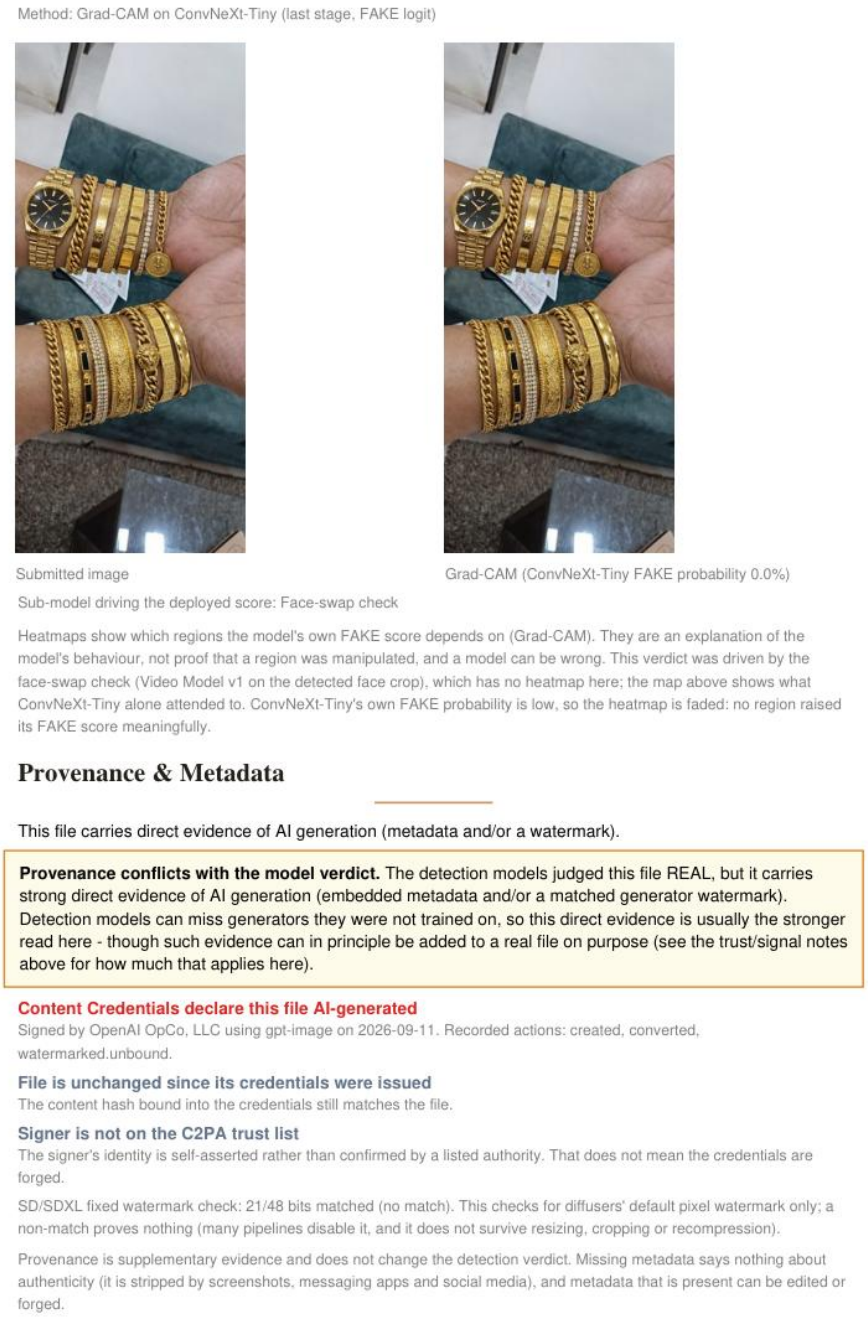


Figure C.4: Generated PDF report for the ChatGPT photograph, page 2: explanation and provenance conflict note

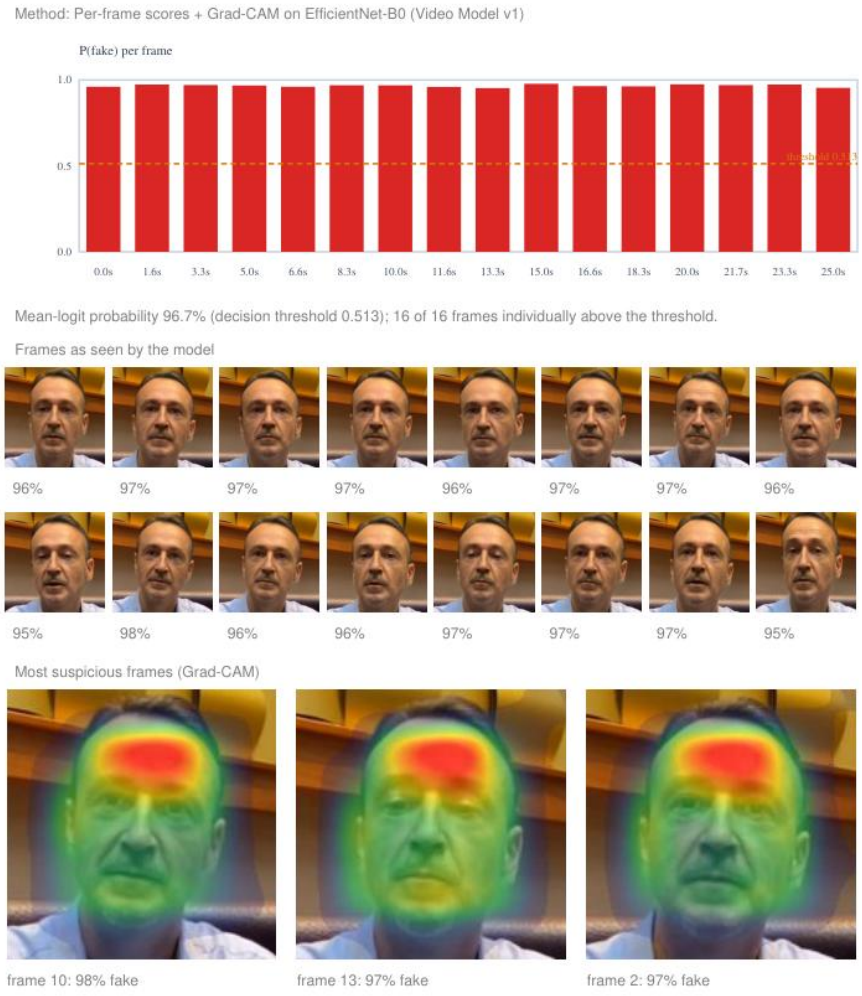


Figure C.5: Generated PDF report for the face-swap video, page 2: per-frame explanation